

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Циклова комісія дисциплін програмної інженерії

КУРСОВА РОБОТА

на тему Розробка гри “Хрестики-нулики”

Виконав: здобувач освіти 2 курсу,
групи 24
спеціальності 121
Інженерія програмного забезпечення

Пічугін Я. С.

(прізвище та ініціали)

Керівник

(підпис)

Олійник В.В.

(прізвище та ініціали)

Полтава – 2024

ЗМІСТ

ВСТУП.....	3
1. ПОСТАНОВКА ЗАВДАННЯ	5
1.1. Основні вимоги до продукту консольного варіанту.....	6
1.2. Основні вимоги до продукту графічного варіанту	7
1.3. Вимоги до інтерфейсу консольного варіанту	9
1.4. Вимоги до інтерфейсу графічного варіанту.....	9
2. ПЛАНУВАННЯ СИСТЕМИ.....	11
2.1. Алгоритм роботи програми консольного варіанту.....	11
2.2. Алгоритм роботи програми графічного варіанту	17
2.3. Тестування консольного варіанту	21
2.4. Тестування графічного варіанту	23
2.5. Інструкція з використання системи консольного варіанту	25
2.6. Інструкція з використання системи графічного варіанту.....	30
ВИСНОВКИ	35
СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ.....	36
ДОДАТОК А. ВИХІДНІ КОДИ	37
ДОДАТОК Б. РЕЗУЛЬТАТИ ТЕСТУВАННЯ.....	38
ДОДАТОК В. ВИМОГИ ДО ІНТЕРФЕЙСУ	60

ВСТУП

Під час мого навчання я здобув базові знання з дисципліни “Основи алгоритмізації та програмування”. Цей курс заклав фундамент для розуміння основних принципів алгоритмізації. В рамках цього курсу я опанував основи мов програмування, що дозволило мені розвинути логічне мислення.

На цьому я не зупинявся та зосередившись на дисципліні “Програмування”. Протягом цього курсу я почав вивчати мову програмування C++. Це дало мені змогу поглибити знання у розробці програмного забезпечення, а також вдосконалити навички написання коду.

Переді мною була поставлена задача розробити гру “Хрестики-нулики”. Для написання гри я використовував мову програмування C++ для створення консольного варіанту програмного продукту. Проте, для реалізації графічного інтерфейсу користувача, я звернувся до мови програмування C# та технології WinForms. Це дало можливість створити зручний та інтуїтивно зрозумілий інтерфейс, що значно покращило користувацький досвід. Для написання коду я використовував програмне забезпечення Visual Studio, яке забезпечило зручну розробку та налагодження програми.

Актуальність обраної теми: Гра “хрестики-нулики” є класичною настільною грою, яка виникла у середині 20-го століття, залишається актуальною і сьогодні.

Гра володіє потенціалом для розвитку мислення. Правила гри є досить простими, для досягнення перемоги потрібно застосовувати стратегічне мислення, аналізувати ходи супротивника і шукати оптимальні стратегії. Це розвиває логічне мислення, увагу, концентрацію і аналітичні здібності.

Метою роботи являється розробка гри “Хрестики-нулики”, яка надасть користувачам можливість насолоджуватися цією класичною грою на своєму комп'ютері.

Для досягнення мети розробки консольної гри “Хрестики-нулики” необхідно виконати наступні завдання:

- розробити алгоритм гри, враховуючи правила “Хрестиків-нуликів”;
- розробити прототип консольного та графічного інтерфейсу застосунку;
- написати програмний код, що забезпечить можливість взаємодії з грою через консоль та графічний застосунок;
- реалізувати логіку гри, включаючи перевірку виграшних комбінацій та завершення гри;
- провести тестування програми для перевірки коректності роботи алгоритмів та виявлення помилок.

Предметом дослідження є розробка гри “Хрестики-нулики” в консольному та графічному середовищах. Цей проєкт передбачає створення програмного продукту, який дозволить користувачам грати в цю гру як в консольному, так і в графічному інтерфейсах. Ключовим завданням є розробка ефективного алгоритму для обробки ходів гравців та визначення переможця в грі, забезпечуючи при цьому зручний та інтуїтивно зрозумілий інтерфейс для користувачів в обох варіантах гри.

Методи дослідження: в процесі виконання курсової роботи були використані наступні методи:

Теоретичні – проведення порівняльного аналізу існуючих версій гри “Хрестики-нулики” з метою вивчення їх структури, геймплею, можливостей розширення функціоналу тощо.

Системний аналіз – детальний розгляд і розподіл гри на складові частини, такі як логіка гри, механіка гри, тощо. Це дозволить зрозуміти, як різні елементи гри взаємодіють між собою і як можна оптимізувати їх роботу.

1. ПОСТАНОВКА ЗАВДАННЯ

В одному містечку, оточеному зеленими лісами, жила дівчинка Надя, яка була прикута до інвалідного візка після нещасного випадку. Вона любила малювати та багато читала. Але зовсім не мала друзів. В холодну пору року їй зовсім не було самотньо, але з приходом теплих та сонячних днів вона дуже засмучувалася, що не має можливості побігати босоніж по зеленій траві, покатися в річці неподалік від її будинку та просто прогулятися разом з батьками рідною вуличкою. Надя часто спостерігала за сусідніми дітлахами, які бавилися. Вони грали в м'яча, стрибали в класики та просто веселилися.

Одного разу Коля зловив на собі погляд Наді, яка сиділа біля вікна і дивилася на нього. Він стояв ніби зачарований і не міг поворушитися. Потім підійшов до будинку і постукав. Двері відчинила мама Наді. Вона запросила його до будинку та провела у кімнату, де була Надя. Ця зустріч стала початком дружби. Друзям було весело разом проводити вільний час, спілкуватися. Проте в них виникла ідея – придумати нову забаву, щоб розважатися.

Надя та Коля вирішили створити свою власну гру – “Хрестики-нулики”. Коля взявся за написання програмного коду, використовуючи середовище розробки Visual Studio, вибравши мови програмування C++ та C#, тоді як Надя зайнялася оформленням дизайну. Програмний продукт було створено у консольному та графічному варіанті. На цей проект вони витратили кілька місяців, вирішуючи різноманітні труднощі та знаходячи вихід разом.

У грі можуть брати участь дві людини, які вибирають, хто буде грати хрестиками, а хто нуликами, і по черзі вводять у вікно консолі за допомогою клавіатури числа, що відповідають клітинкам, або, якщо обрано графічний варіант, натискаючи мишею на відповідний квадрат на ігровому полі. Виграє

та людина, яка перша складе виграшну комбінацію з прямої з прямої з хрестиків або нуликів по горизонталі, по вертикалі або по діагоналі.

Ця гра стала не лише засобом розваги, а й зв'язком, що ще більше зміцнив дружбу між ними. Вони розуміли, що незалежно від результату, головне це час, який вони проводять разом.

1.1. Основні вимоги до продукту консольного варіанту

Гра повинна бути написана на мові програмування C++.

ФУНКЦІОНАЛЬНІ ВИМОГИ:

- гра має підтримувати двох гравців, які грають по черзі;
- гравці мають мати можливість вибрати, чим вони будуть грати: хрестиками або нуликами;
- гра має відображати ігрове поле в консолі;
- гравці мають вводити номер комірки, в яку вони хочуть зробити свій хід, використовуючи клавіатуру;
- гра повинна перевіряти, чи не зайнята введена комірка, і якщо зайнята, то сповістити гравця ввести ще раз свій хід;
- гра повинна перевіряти після кожного ходу, на наявність виграшної комбінації з трьох хрестиків або трьох нуликів по горизонталі, вертикалі або діагоналі;
- гра повинна відобразити повідомлення про виграшну комбінацію чи нічию.

НЕФУНКЦІОНАЛЬНІ ВИМОГИ:

- інтерфейс гри повинен бути простим та чітким, з мінімальною кількістю непотрібних елементів;
- гра повинна працювати стабільно без збоїв та зависань;
- всі дії гравців повинні оброблятися майже миттєво, без помітних затримок.

У ході розробки програмного продукту було використано середовище

розробки Visual Studio та мову програмування C++. Продукт містить в собі такі бібліотеки: “iostream”, “Windows.h” та “vector”.

ПЕРЕЛІК РЕАЛІЗОВАНИХ ПІД ЧАС ВИКОНАННЯ КУРСОВОЇ РОБОТИ ФУНКЦІОНАЛЬНИХ ВИМОГ

- реалізовано можливість для двох гравців грати в гру по черзі;
- реалізовано можливість для гравців вибрати, чим вони будуть грати: хрестиками або нуликами;
- реалізовано відображення ігрового поля в консолі;
- реалізовано введення гравцями номера комірки, в яку вони хочуть зробити свій хід, використовуючи клавіатуру;
- реалізовано перевірку, чи не зайнята введена комірка, і якщо зайнята, то просити гравця ввести ще раз;
- реалізовано перевірку після кожного ходу на виграшну комбінацію з хрестиків або нуликів по горизонталі, вертикалі або діагоналі;
- реалізовано відображення повідомлення про виграшну комбінацію або нічию;
- реалізовано запит на реванш гри.

ПЕРЕЛІК НЕРЕАЛІЗОВАНОЇ ФУНКЦІОНАЛЬНОСТІ

- не реалізовано збереження гри та її подальшого відновлення;
- не реалізовано можливість грати з комп'ютером;
- не реалізовано можливість зміни розміру ігрового поля;
- не реалізовано можливість ведення статистики ігор.

Перелік нереалізованих функцій пов'язаний з недостатнім рівнем знань та складністю їх впровадження. Відсутність певних навичок у конкретних областях програмування може створювати перешкоди для реалізації певних задач.

1.2. Основні вимоги до продукту графічного варіанту

Гра повинна бути написана на мові програмування C#.

ФУНКЦІОНАЛЬНІ ВИМОГИ:

- гра повинна бути для двох гравців, які ходять по черзі;
- гравці повинні мати вибір вибрати символи для ходів;
- відображення поточного гравця;
- перевірка на виграшну комбінацію або на нічию після кожного ходу;
- виграшна комбінація повинна підсвічуватись;
- гра повинна мати можливість зробити рестарт.

НЕФУНКЦІОНАЛЬНІ ВИМОГИ:

- гра повинна мати інтуїтивно зрозумілий інтерфейс користувача;
- гра повинна працювати без помилок та збоїв.

У ході розробки програмного продукту було використано середовище розробки Visual Studio та мову програмування C#. В продукті використовується Windows Forms, яка є частиною .NET Framework та бібліотеки:

“System”, “System.Collections.Generic”,
 “System.ComponentModel”, “System.Data”, “System.Drawing”, “System.Linq”,
 “System.Text”, “System.Threading.Tasks”, “System.Windows.Forms”.

ПЕРЕЛІК РЕАЛІЗОВАНИХ ПІД ЧАС ВИКОНАННЯ КУРСОВОЇ РОБОТИ ФУНКЦІОНАЛЬНИХ ВИМОГ

- реалізована гра для двох гравців;
- реалізовано вибір символів для гравців;
- реалізовано відображення поточного гравця;
- реалізовано можливість натискання на кнопки для здійснення ходу;
- реалізовано перевірку після кожного ходу на виграшну комбінацію з хрестиків або нуликів по горизонталі, вертикалі або діагоналі;
- реалізовано підсвічування виграшної комбінації;
- реалізовано відображення повідомлення про виграшну комбінацію або нічию;
- реалізовано запит на продовження гри після закінченої партії.

ПЕРЕЛІК НЕРЕАЛІЗОВАНОЇ ФУНКЦІОНАЛЬНОСТІ

- не реалізовано можливість грати з комп'ютером;

- не реалізовано можливість ведення статистики ігор;
- не реалізовано можливість зміни розміру ігрового поля;
- не реалізовано можливість використання звукових ефектів;
- не реалізовано можливість використовувати гру для повноекранного режиму.

Перелік нереалізованих функцій пов'язаний з недостатнім рівнем знань та складністю їх впровадження. Недостатні навички у певних галузях програмування можуть створювати перешкоди для реалізації конкретних задач.

1.3. Вимоги до інтерфейсу консольного варіанту

Вимоги до зовнішнього вигляду програми:

- простота та зрозумілість інтерфейсу;
- чітка візуалізація ігрового поля;
- зручне керування.

Оскільки код програми написаний на C++, то GUI-фреймворк не використовується. Візуальні елементи та взаємодія з користувачем реалізовані за допомогою стандартних засобів консольного вводу/виводу Windows.

Програма запускається в консолі Windows. Вікно програми містить:

- заголовок;
- меню;
- ігрове поле;
- підказка про наступні дії користувача.

Прототип інтерфейсу був розроблений у програмному забезпеченні Pencil GUI (додаток B.1).

1.4. Вимоги до інтерфейсу графічного варіанту

Вимоги до зовнішнього вигляду програми:

- проста та інтуїтивно зрозуміла навігація;

- приємний візуальний дизайн з використанням не дратівливих кольорів;
- інтерфейс має відображати поточного гравця.

Програма розроблена на мові програмування C# з використанням GUI-фреймворку Windows Forms в середовищі Visual Studio. Для створення форми був використаний візуальний конструктор Windows Forms.

Програма запускається у графічному вікні та містить такі елементи:

- ігрове поле;
- надпис про поточного гравця;
- кнопки управління гри.

Прототип інтерфейсу був розроблений у програмному забезпеченні Pencil GUI (додаток В.2).

2. ПЛАНУВАННЯ СИСТЕМИ

2.1. Алгоритм роботи програми консольного варіанту

Запускаємо програму. Після запуску програми, з'являється меню. Користувач повинен ввести номер пункту меню, який він бажає вибрати. У меню є три варіанти для вибору:

- виведення правил програми – якщо користувач вибирає цей пункт, то програма виводить правила роботи програми та повертається до початкового стану, а саме очікування на введення номера пункту меню;
- запуск програми – якщо користувач вибирає цей пункт, то програма запускається. Після закінчення програми користувач має можливість продовжити роботу програми або завершити її. Якщо користувач вибирає варіант “Так”, то програма повертається до початкового стану, а саме запуск програми. Якщо користувач вибирає варіант “Ні”, то програма завершується і переходить до пункту “Кінець”;
- завершення програми – якщо користувач вибирає цей пункт, то програма завершується і переходить до пункту “Кінець”.

Алгоритм роботи меню програмного продукту зображений блок-схемою на рисунку 2.1.

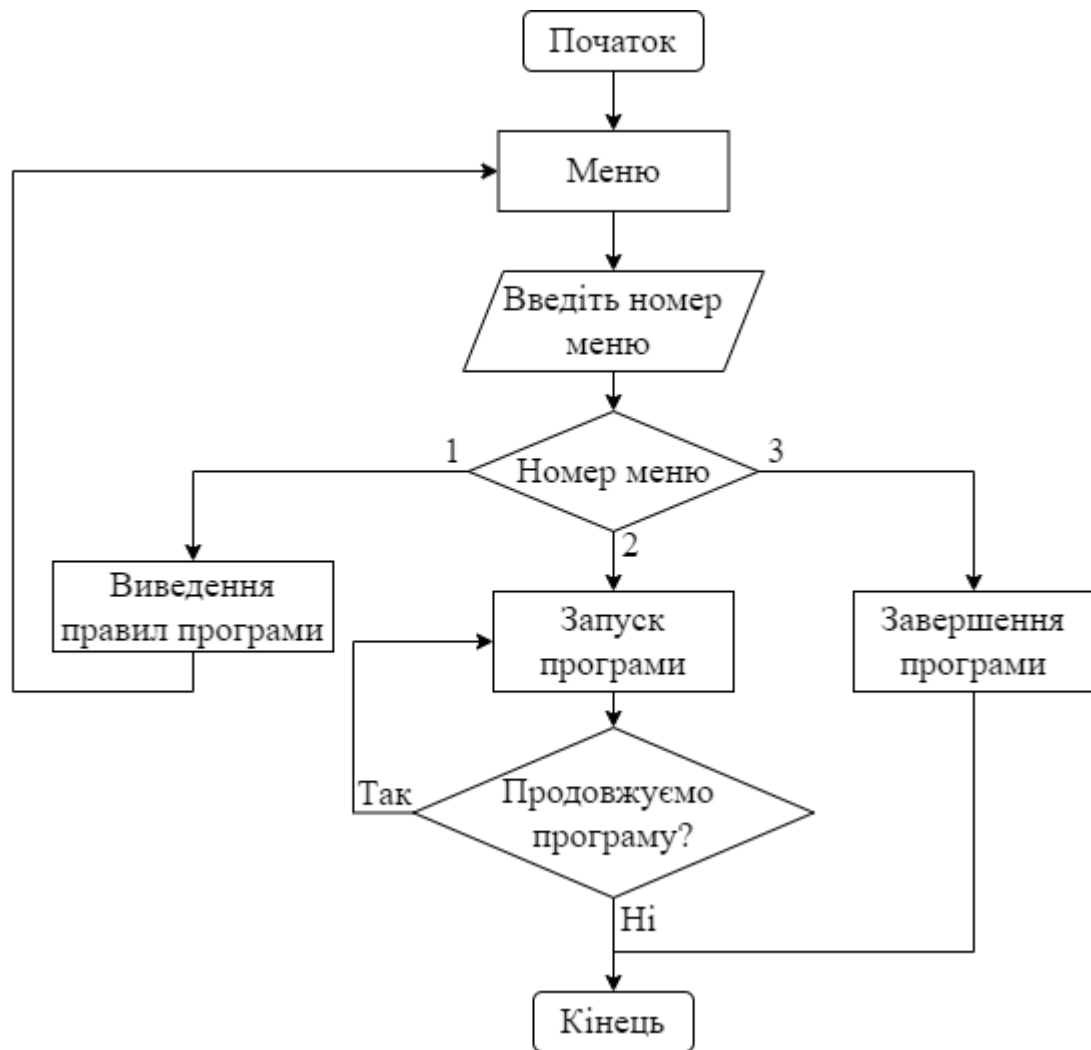


Рисунок 2.1 – Робота меню програмного продукту

Функція `board_out` відображає двовимірний вектор символів у вигляді таблиці 3x3. Кожен елемент вектора відображається у вигляді символу, якщо він не є пробілом, або у вигляді числа, якщо вказано виводити числа (`show_numbers == true`), або у вигляді пробілу, якщо вказано не виводити числа (`show_numbers == false`).

Функція використовує два вкладених цикли для перебору елементів вектора. Перший цикл (по змінній `i`) перебирає рядки, а другий цикл (по змінній `t`) перебирає стовпці. Перед виведенням кожного елемента, крім першого в рядку, функція виводить вертикальну лінію `"|"` для відокремлення стовпців.

Якщо елемент є пробілом, то функція перевіряє значення параметра `show_numbers`. Якщо `show_numbers == true`, то функція виводить число, яке відповідає поточній позиції елемента в векторі ($i * 3 + t + 1$), в іншому випадку функція виводить пробіл.

Після виведення кожного рядка, крім останнього, функція виводить горизонтальні лінії `---|---|---` для відокремлення рядків.

Таким чином, функція відображає вектор у вигляді таблиці 3x3, де кожен елемент відображається у вигляді символу або числа, та розділений лініями для зручності в сприйнятті.

Алгоритм роботи функції `board_out` зображено блок-схемою на рисунку 2.2.

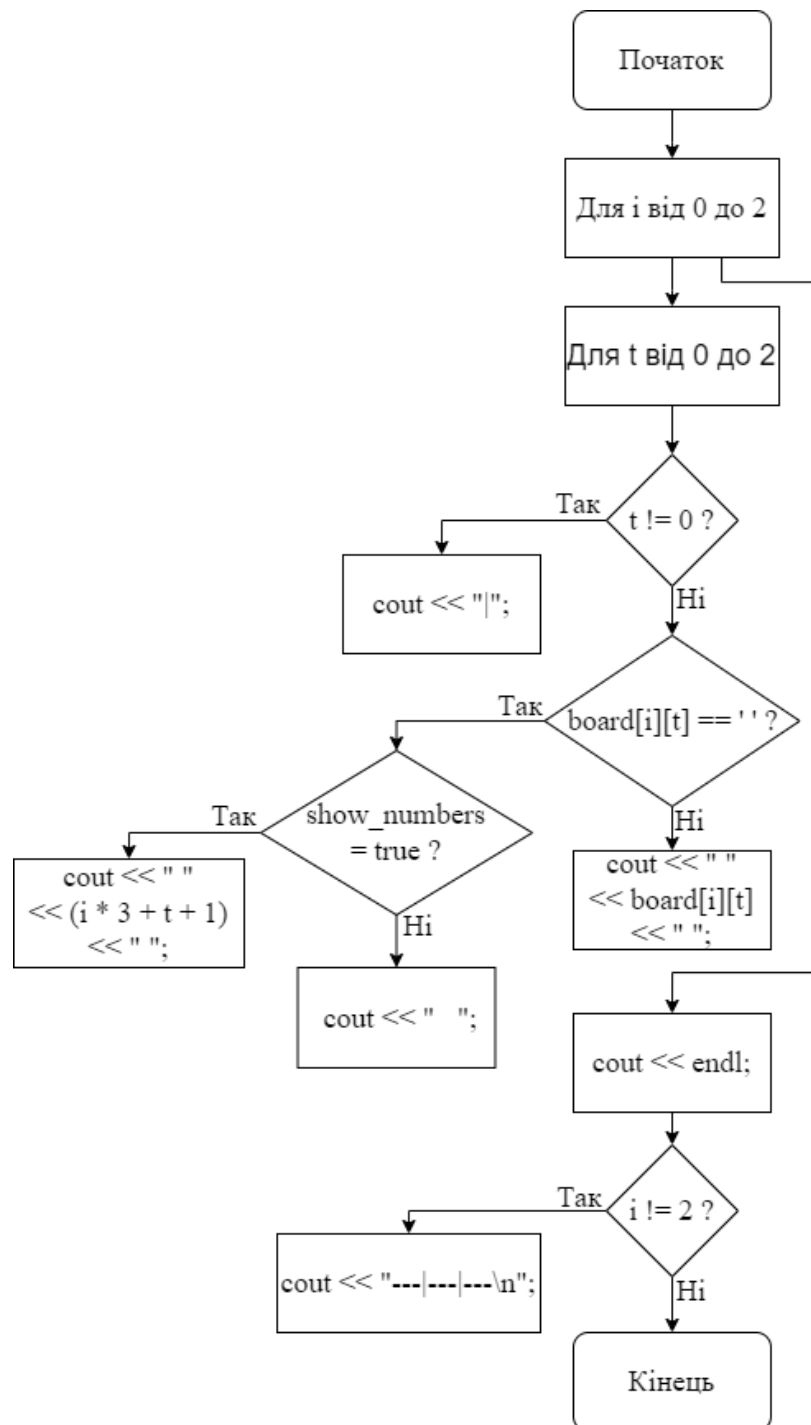


Рисунок 2.2 – Алгоритм функції board_out

Функція board_out виводить гральну дошку на екран. Функція приймає два параметри: вектор векторів board та булевий параметр show_numbers. Далі створюється гральна дошка розміром 3x3.

Гравці обирають символи, які будуть використовуватися в грі. Гравець 1 вводить свій символ (X або O), після чого відбувається перевірка на

коректність введення. Символ гравця 2 визначається автоматично, в залежності від символу гравця 1.

Перед початком гри виводиться дошка з номерами клітинок. Гравець, (`currentPlayer`), вводить номер клітинки, після чого відбувається перевірка на коректність введення. Якщо введення коректне, то на дошці відмічається символ гравця.

Після кожного ходу перевіряються рядки, стовпці та діагоналі на наявність переможця. Якщо є переможець, виводиться повідомлення про перемогу та питання, чи грати далі. Якщо нічия, виводиться повідомлення про нічию та питання, чи грати далі. Якщо гра продовжується, то змінюється хід наступного гравця.

Гра триває доти, доки не буде визначено переможця або не буде зафіксовано нічию. У коді використовуються цикли `for` для перебору клітинок дошки, цикл `while` для перевірки коректності введення символу поточного гравця (`currentPlayer`) та цикл `while (true)` для головного циклу гри. Також використовуються умовні конструкції `if` для перевірки умов та оператор `break` для виходу з циклу.

Алгоритм роботи програмного продукту зображено блок-схемою на рисунку 2.3 та рисунку 2.4.

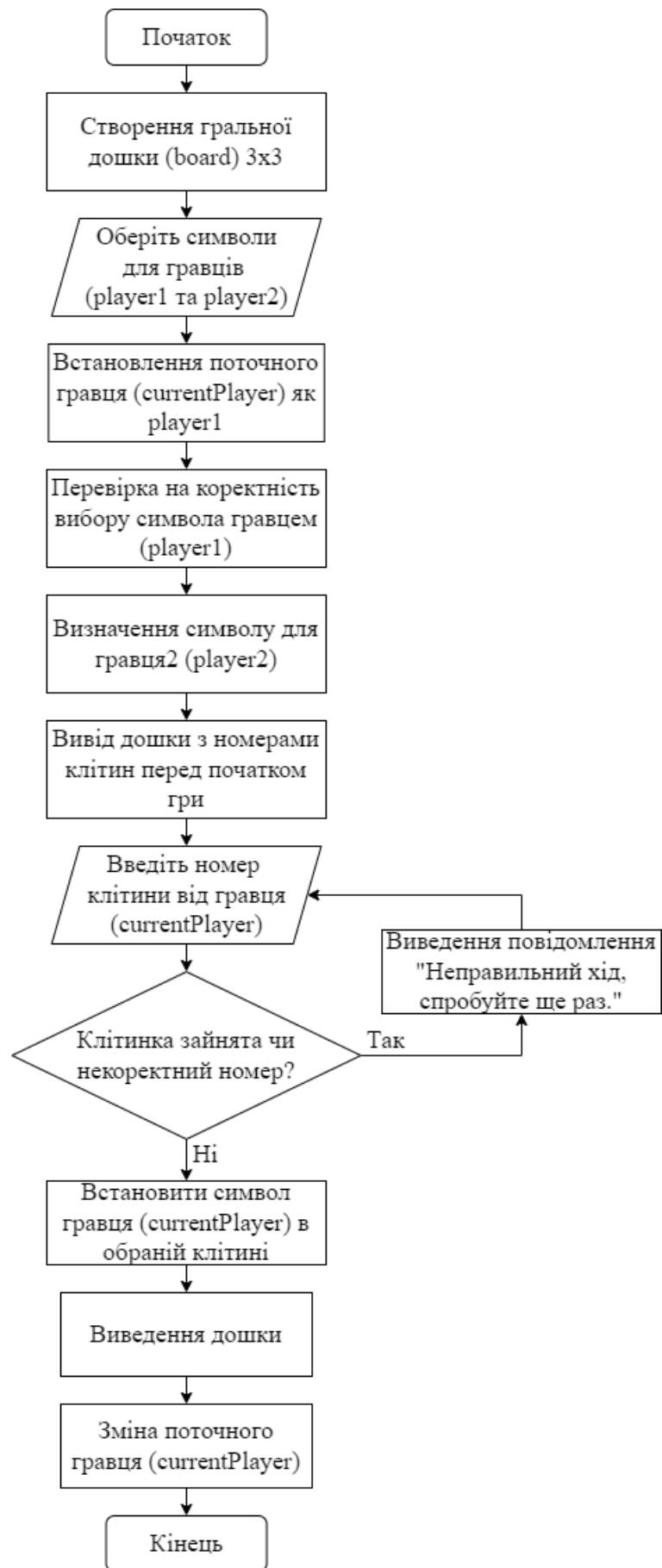


Рисунок 2.3 – Робота програмного продукту

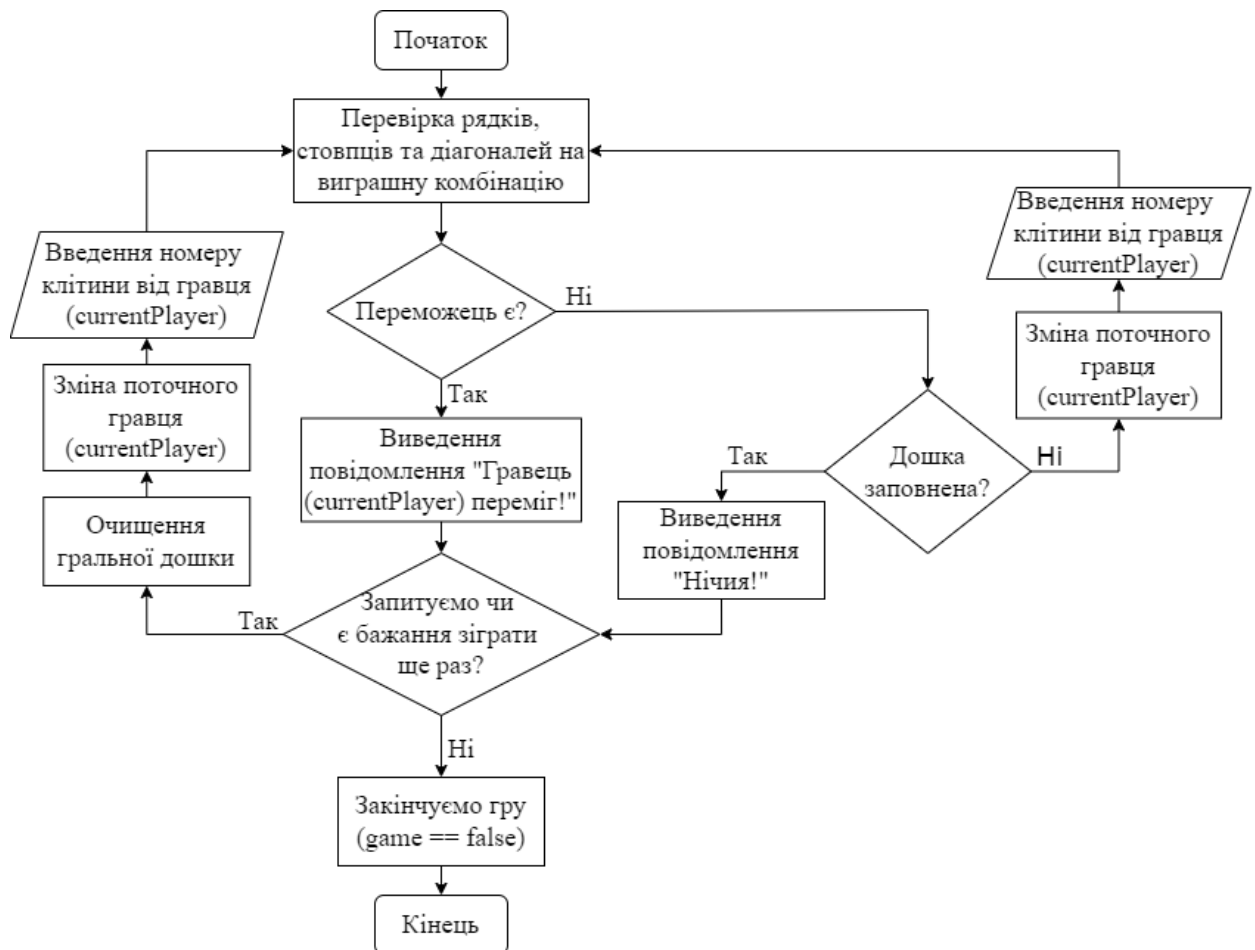


Рисунок 2.4 – Робота програмного продукту

2.2. Алгоритм роботи програми графічного варіанту

Після запуску програми з'являється форма Menu. У користувача є три варіанти для вибору:

- правила – після натискання на кнопку “Правила” поточна форма Menu приховується та з'являється інша форма Rules. Прочитавши правила гри, гравець має вибір повернутися в меню або одразу почати гру;
- грати в гру – після натискання на кнопку “Грати в гру” поточна форма Menu приховується та з'являється інша форма Game. Після завершення партії гри користувач має вибір продовжити гру чи завершити. Якщо користувач вибирає продовжити гру, то дошка очищається. Якщо користувач вибирає завершити гру, то гра завершується;
- завершити гру – після натискання на кнопку “Завершити гру”, то поточна форма закривається.

Алгоритм роботи меню програмного продукту зображений блок-схемою на рисунку 2.5.

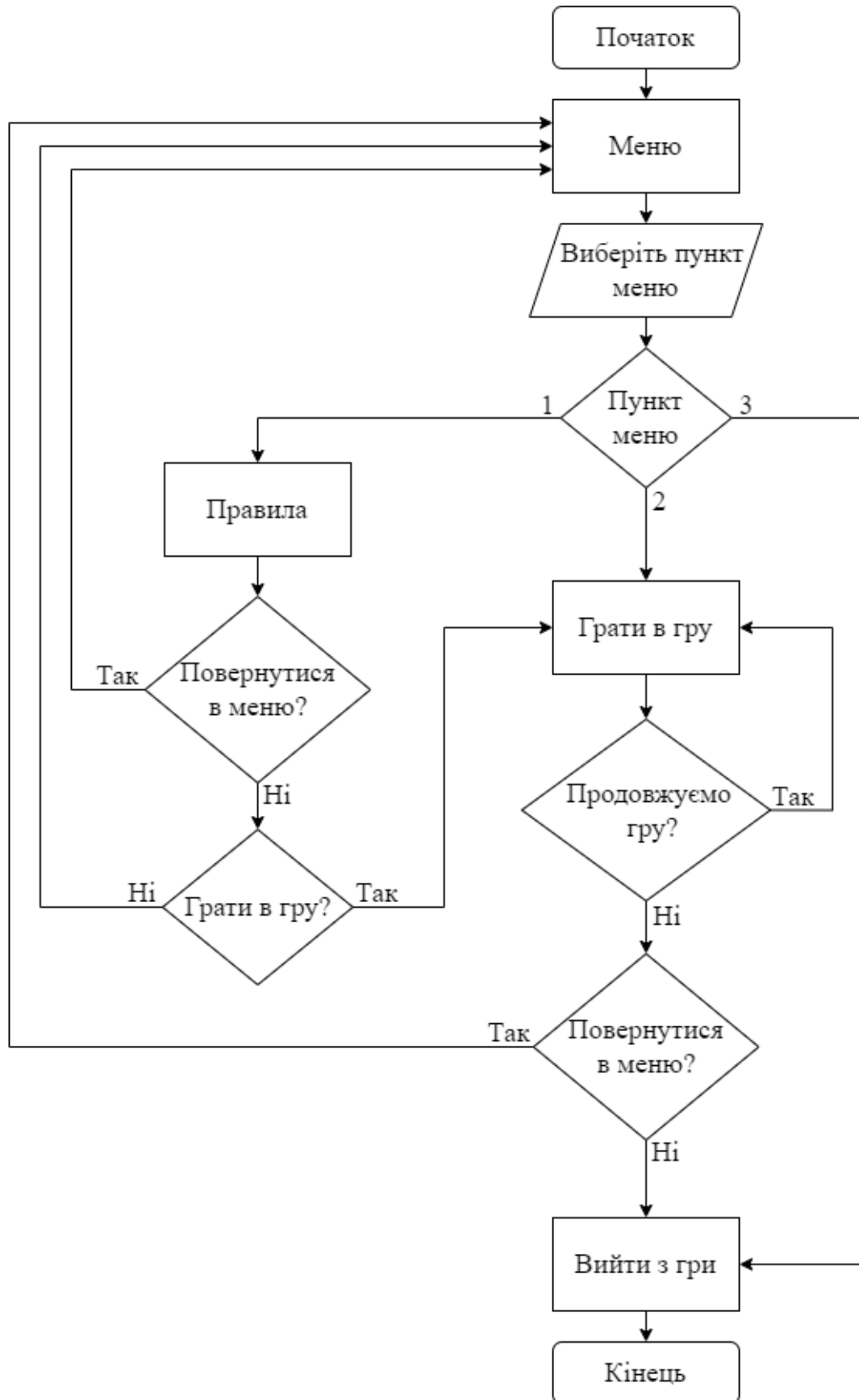


Рисунок 2.5 – Робота меню програмного продукту

Коли користувач натискає на кнопку “Грати в гру” відкривається форма Game та ще 1 діалогове вікно. У цьому діалоговому вікні гравець 1 повинен вибрати бажаний символ (X або O), після чого програма автоматично присвоює протилежний символ гравцю 2 та закриває вікно вибору символів. Після вибору символів з’являється головне вікно гри, а саме гральна дошка розміром 3x3. Перший хід надається гравцю 1.

У грі кожна комірка представлена кнопкою. При натисканні на кнопку перевіряється, чи порожня вона. Якщо так, у текст кнопки записується символ поточного гравця, а символ також записується у відповідну комірку в масиві board. Після кожного ходу відбувається перевірка на наявність виграшної комбінації. Якщо виграшна комбінація знайдена, то ці комірки підсвічуються, і виводиться повідомлення про перемогу. Якщо ж всі комірки заповнені і виграшної комбінації немає, виводиться повідомлення про нічию. Після цього викликається вікно із запитом, чи хочуть гравці зіграти ще раз. Якщо гравці обирають "Так", то гра скидається, очищаючи ігрове поле і масив, що зберігає інформацію про зайняті комірки. Після рестарту гри змінюється черговість гравців. Якщо гравці обирають "Ні" - програма закривається.

Для перевірки виграшної комбінації програма проходить по горизонтальних, вертикальних та діагональних лініях, перевіряючи наявність трьох однакових символів у ряд. Якщо така комбінація знайдена, повертається масив виграшних комірок, інакше - null. Для перевірки на нічию програма перевіряє, чи всі комірки заповнені. Якщо так, повертається true, інакше - false.

Алгоритм роботи програмного продукту зображено блок-схемою на рисунку 2.6 та рисунку 2.7.

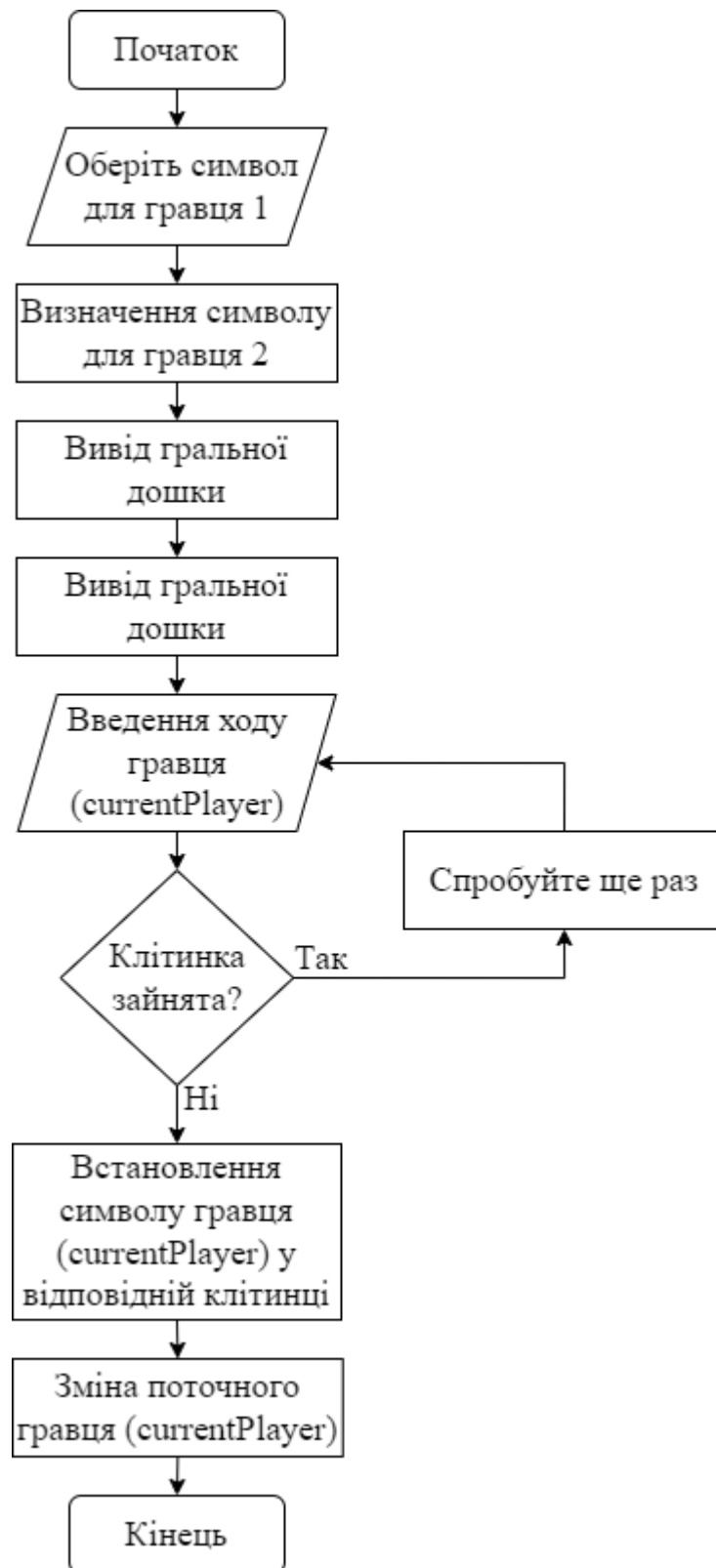


Рисунок 2.6 – Робота програмного продукту

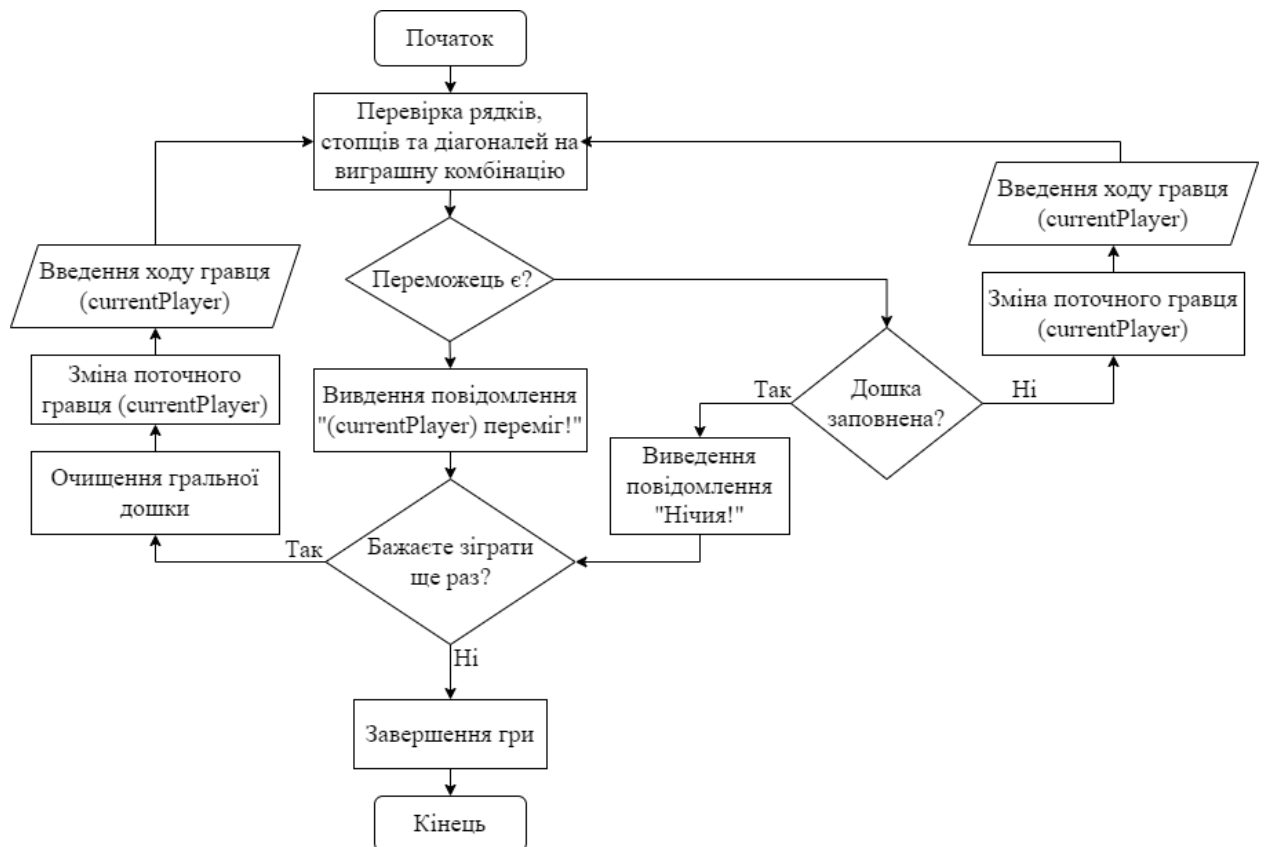


Рисунок 2.7 – Робота програмного продукту

2.3. Тестування консольного варіанту

У ході тестування було обрано 3 методи: black box, white box та unit.

Першим методом тестування буде “Чорна скринька”. Це метод тестування, при якому тестувальник перевіряє функціональність програми, не заглиблюючись у її внутрішню структуру або код.

Нижче наведено опис тестування чорної скриньки для програми “Хрестики-нулики”:

- запуск програми. Після запуску програми має з'явитися головне меню з трьома пунктами: “Мета та правила гри”, “Розпочати гру” та “Закінчити гру” (додаток Б.1);
- перевірка пункту “Мета та правила гри”. Після вибору пункту “1” програма повинна вивести мету та правила гри (додаток Б.2);
- перевірка пункту “Розпочати гру”. Після вибору пункту “2” має з'явитися запит на вибір символу для гравця 1 (X або O). Програма повинна

перевіряти коректність введеного символу і не приймати інші символи (додаток Б.3);

- перевірка гри. Вводимо номери клітинок (від 1 до 9). Програма повинна перевіряти коректність ходу, не дозволяти займати вже зайняті клітинки або вводити номери поза межами 1-9 (додаток Б.4);
- перевірка умов перемоги. Програма повинна коректно визначати переможця, коли три символи розташовані по горизонталі, вертикалі або діагоналі (додаток Б.5, додаток Б.6 та додаток Б.7);
- перевірка нічий. Програма повинна коректно визначати ситуацію, коли всі клітинки зайняті і жоден з гравців не має трьох символів в ряд (додаток Б.8);
- перевірка некоректного вибору в меню. Програма повинна коректно обробляти некоректні введення в головному меню (вибір пункту, який не існує) і виводити відповідне повідомлення (додаток Б.9).

Другим методом тестування буде “Біла скринька”. Це метод тестування, який перевіряє внутрішнє функціонування системи. У цьому методі тестування перевіряються внутрішня структура, будова і код програмного забезпечення.

Нижче наведено опис тестування чорної скриньки для програми “Хрестики-нулики”:

- перевірка функції `board_out`. Перевіримо чи правильно відображаються номери клітинок, коли `show_numbers` встановлено `true` і коли `false` (додаток Б.10 та додаток Б.11);
- перевірка на правильність обробки вводу номерів пункту (додаток Б.12);
- цикл гри. Перевіряється, чи коректно програма здійснює перехід ходу між гравцями, правильно обробляє введення номера клітинки і відображає оновлену дошку після кожного ходу (додаток Б.13);
- перевірка чи правильно визначаються виграшні комбінації по горизонталі, вертикалі та діагоналях. Перевіряється, чи коректно обробляється випадок

нічиєї, коли всі клітинки зайняті і жоден з гравців не має виграшної комбінації (додаток Б.14, додаток Б.15, додаток Б.16 та додаток Б.17);

— перевірка очищення дошки для нової гри (додаток Б.18);

Третім методом тестування буде “Unit”, який проводився за допомогою фреймворку “Google test”. Unit тестування – це метод програмного забезпечення, при якому окремі модулі коду перевіряються на коректність їх роботи.

Розіб'ємо тестування на частини:

— перевіримо правильність виводу меню програмного продукту (додаток Б.19);

— перевіримо правильність виведення гральної дошки (додаток Б.20);

— перевіримо на правильність функцію для перевірки перемоги, коли є виграшна комбінація, інакше нічия (додаток Б.21);

— перевіримо на правильність функції для перевірки нічії (додаток Б.22);

— перевіримо на правильність самої гри (додаток Б.23).

У ході тестування трьома методами консольного додатку виявлено помилок програми не було.

2.4. Тестування графічного варіанту

У ході тестування було обрано 3 методи: black box, white box та unit.

Першим методом тестування буде “Чорна скринька”.

Тестування чорної скриньки – це метод тестування, при якому тестувальник перевіряє функціональність програми, не заглиблюючись у її внутрішню структуру або код.

Нижче наведено опис тестування чорної скриньки для програми “Хрестики-нулики”:

— запуск програми. Перевіряється, чи програма відкриває головне меню без помилок і чи всі кнопки відображаються правильно (додаток Б.24);

- натискання на кнопку “Правила”. Після натискання кнопки відкривається форма з правилами гри. Перевіряється, чи ця форма відображається коректно, і чи головне меню приховується (додаток Б.25);
- натискання на кнопку “Грати”. Після натискання кнопки відкривається форма гри. Перевіряється, чи ця форма відображається коректно, і чи головне меню приховується. Далі перевіряється, чи з'являється діалогове вікно для вибору символу гравця 1 (Х або О), і чи працюють кнопки вибору символу (додаток Б.26 та додаток Б.27);
- перевірка виграшу. Перевіряється, чи програма правильно визначає виграшну комбінацію по горизонталі, вертикалі та діагоналі. Після завершення гри перевіряється, чи з'являється повідомлення про перемогу відповідного гравця і чи підсвічуються виграшні комірки (додаток Б.28, додаток Б.29, додаток Б.30);
- перевірка нічий. Перевіряється, чи програма правильно визначає ситуацію, коли всі комірки заповнені і немає виграшної комбінації. Після завершення гри перевіряється, чи з'являється повідомлення про нічию (додаток Б.31);
- запит на повторну гру. Після завершення гри перевіряється, чи з'являється діалогове вікно з питанням про продовження гри (додаток Б.32);

Другим методом тестування буде “Біла скринька”.

Тестування білої скриньки – це метод тестування, який перевіряє внутрішнє функціонування системи. У цьому методі тестування перевіряються внутрішня структура, будова і код програмного забезпечення.

Нижче наведено опис тестування чорної скриньки для програми “Хрестики-нулики”:

- ознайомимось з кодом форми меню і перевіримо, чи коректно працюють кнопки навігації до інших форм (додаток Б.33, додаток Б.34, додаток Б.35 та додаток Б.36);

- ознайомимось з кодом форми правил і перевіримо, чи коректно працюють кнопки навігації до інших форм (додаток Б.37, додаток Б.38, додаток Б.39 та додаток Б.40);
- ознайомимось з кодом форми гри і перевіримо, чи коректно працюють функції (додаток Б.41, додаток Б.42, додаток Б.43, додаток Б.44, додаток Б.45, додаток Б.46 та додаток Б.47).

Третім методом тестування буде “Unit”, який проводився за допомогою фреймворку “MStest”. Unit тестування в – це метод програмного забезпечення, при якому окремі модулі коду перевіряються на коректність їх роботи.

Розіб’ємо тестування на частини:

- перевірка на правильність виконання кнопок форми меню (додаток Б.48);
- перевірка на правильність виконання кнопок форми правил (додаток Б.49);
- перевірка на правильність вибору символів, натискання кнопок-комірок, виграшної комбінації, нічий, підсвічування виграшних комірок, відображення вікна із запитом про продовження гри, методу скидання гри, методу для чергування ходів та відображення поточного гравця (додаток Б.50).

У ході тестування трьома методами графічного додатку виявлено помилок програми не було.

2.5. Інструкція з використання системи консольного варіанту

Запуск програмного продукту виконується через файл Course-Work.exe (рисунок 2.8).

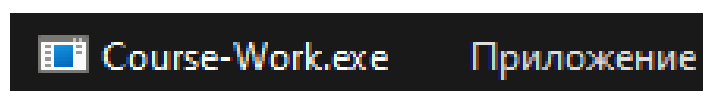


Рисунок 2.8 – Файл гри

Після запуску файлу Course-Work.exe в нас відкривається консоль з головним меню гри (рисунок 2.9).

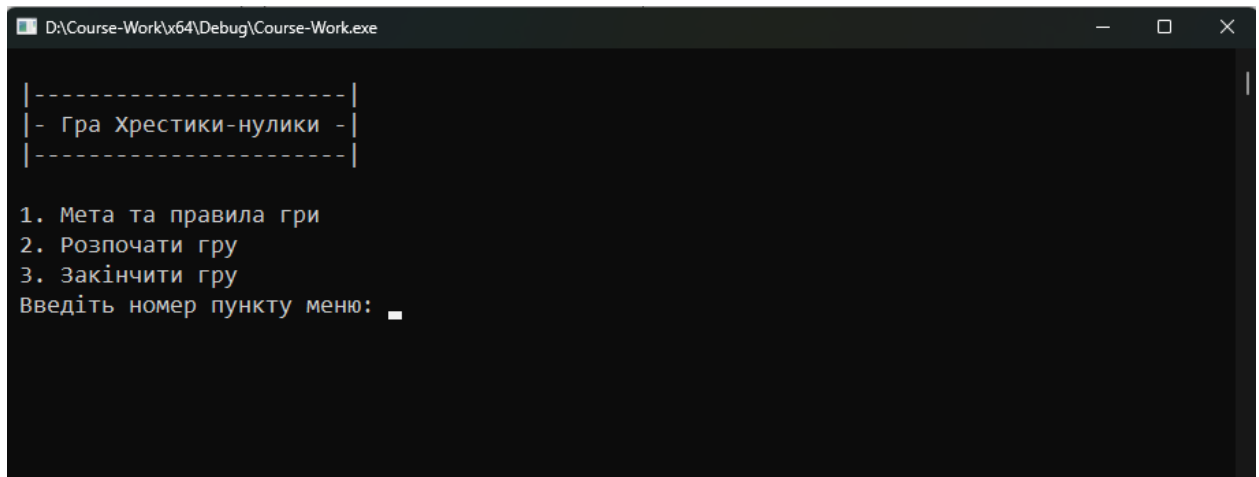


Рисунок 2.9 – Головне меню програмного продукту

При веденні з клавіатури номера пункту меню 1 ми побачимо правила гри (рисунок 2.10).

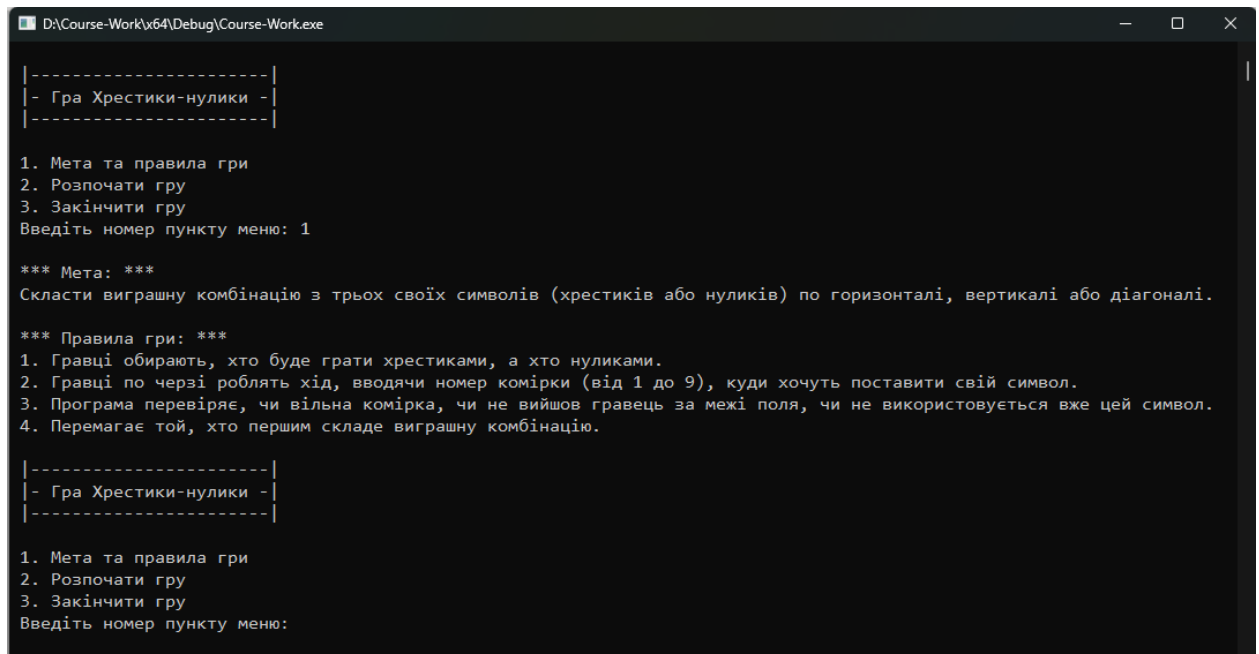


Рисунок 2.10 – Правила гри

При веденні з клавіатури номера пункту меню 2 ми запустимо саму гру (рисунок 2.11). Де програма просить користувача ввести символ гравця 1.

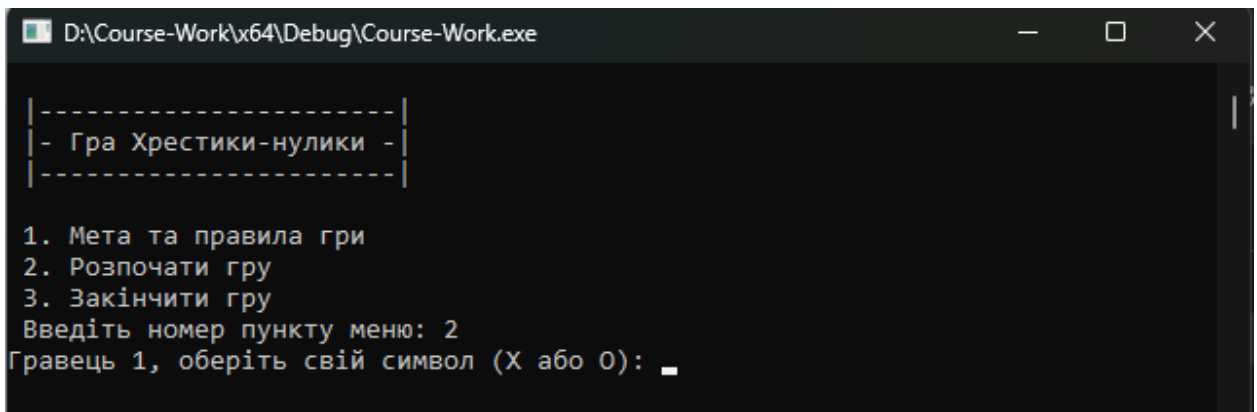


Рисунок 2.11 – Програма просить вибрати символ для гравця 1

Щоб обрати символ потрібно вибрати англійську розкладку клавіатури та ввести велику літеру “X” або “O” (рисунок 2.12).

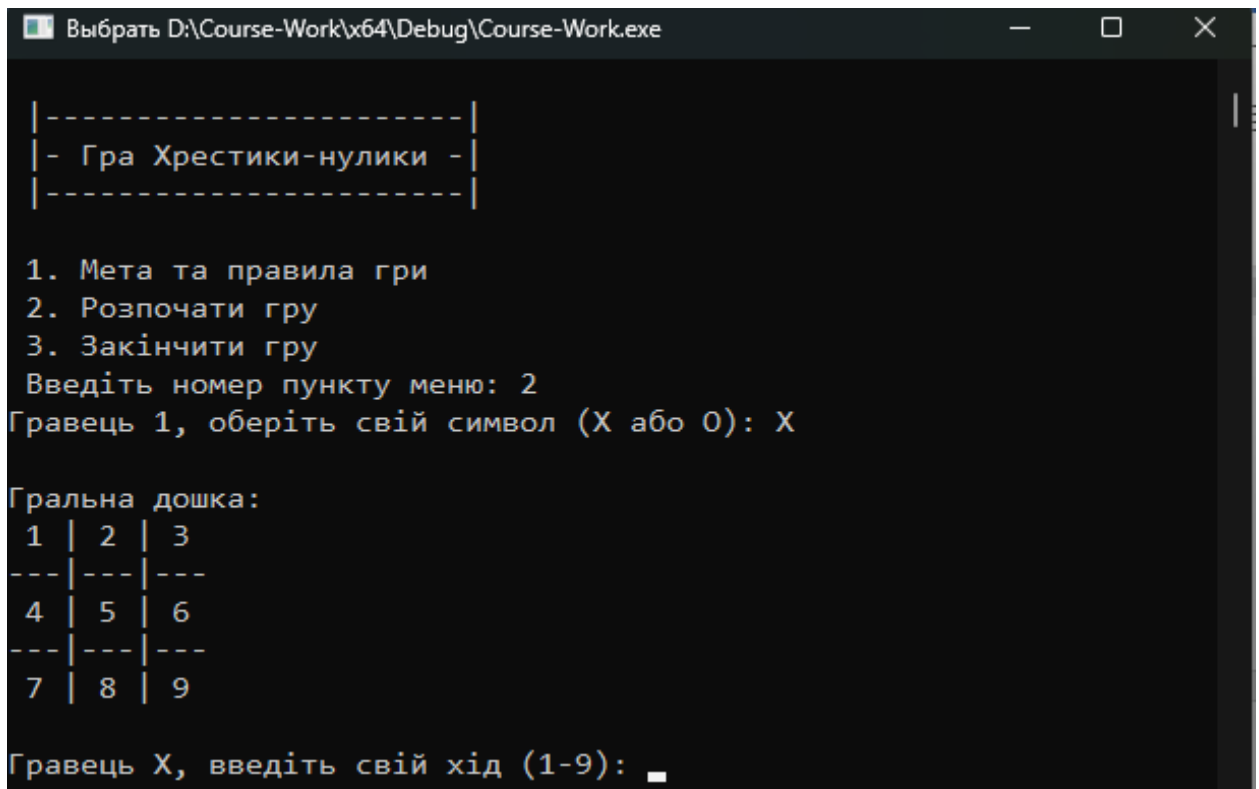


Рисунок 2.12 – Вибір символу для гравця 1

В іншому випадку, якщо буде неправильно введено символ для гравця 1, то буде виведено помилку (рисунок 2.13).

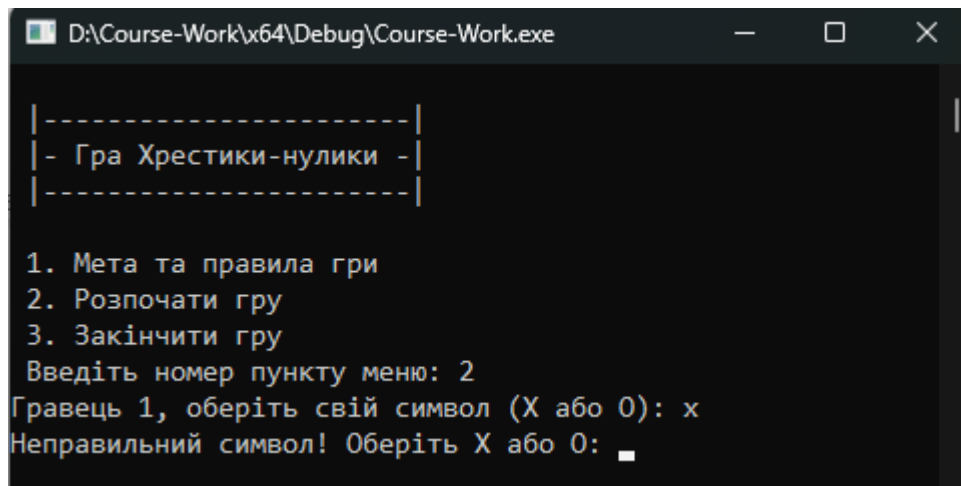


Рисунок 2.13 – Повідомлення про неправильний вибір символу

Після вибору символу для гравця 1, гра автоматично призначає протилежний символ для гравця 2 в залежності від символу гравця 1.

Коли гравці обрали символи виводиться дошка з номерами клітин (рисунок 2.14).

Гральна дошка:

1	2	3
---	---	---
4	5	6
---	---	---
7	8	9

Рисунок 2.14 – Номера клітин гральної дошки

Після чого можна вносити свої ходи в гральну дошку за допомогою клавіатури вводючи цифри від 1 до 9 (рисунок 2.15).

```

Гравець X, введіть свій хід (1-9): 1
X |  | 
---|---|---
  |  | 
---|---|---
  |  | 
Гравець O, введіть свій хід (1-9): 2
X | O | 
---|---|---
  |  | 
---|---|---
  |  | 
Гравець X, введіть свій хід (1-9): 3
X | O | X
---|---|---
  |  | 
---|---|---
  |  | 
Гравець O, введіть свій хід (1-9): 4
X | O | X
---|---|---
O |  | 
---|---|---
  |  | 
Гравець X, введіть свій хід (1-9): 5
X | O | X
---|---|---
O | X | 
---|---|---
  |  | 
Гравець O, введіть свій хід (1-9): 6
X | O | X
---|---|---
O | X | O
---|---|---
  |  | 
Гравець X, введіть свій хід (1-9): 7
X | O | X
---|---|---
O | X | O
---|---|---
X |  | 
Гравець X переміг!
Грати ще раз? (Так/Ні):_

```

Рисунок 2.15 – Внесені ходи за допомогою клавіатури

Якщо під час гри виявлено виграшну комбінацію або нічию буде виведено запитання про запит чи хочуть гравці зіграти ще раз, якщо гравці виберуть “Так”, то гру буде перезапущено та першим буде ходити гравець протилежного символу (рисунок 2.16).

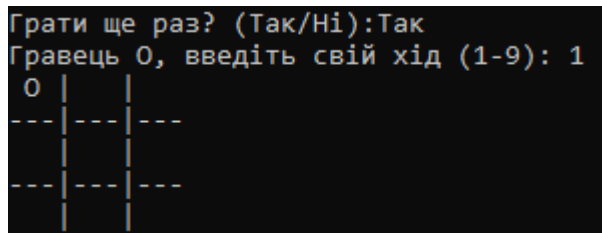


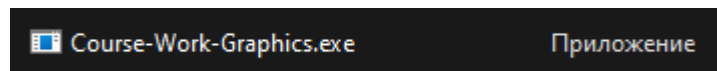
Рисунок 2.16 – Перезапуск гри

В іншому випадку, якщо гравці виберуть “Ні”, то гру буде завершено і консоль закриється.

При введенні з клавіатури номера пункту меню 3 гру буде завершено.

2.6. Інструкція з використання системи графічного варіанту

Запуск програмного продукту виконується через файл Course-Work-Graphics.exe (рисунок 2.17).



Рисунку 2.17 – Файл гри

Після запуску файлу Course-Work-Graphics.exe в нас відкривається вікно з головним меню гри (рисунок 2.18).

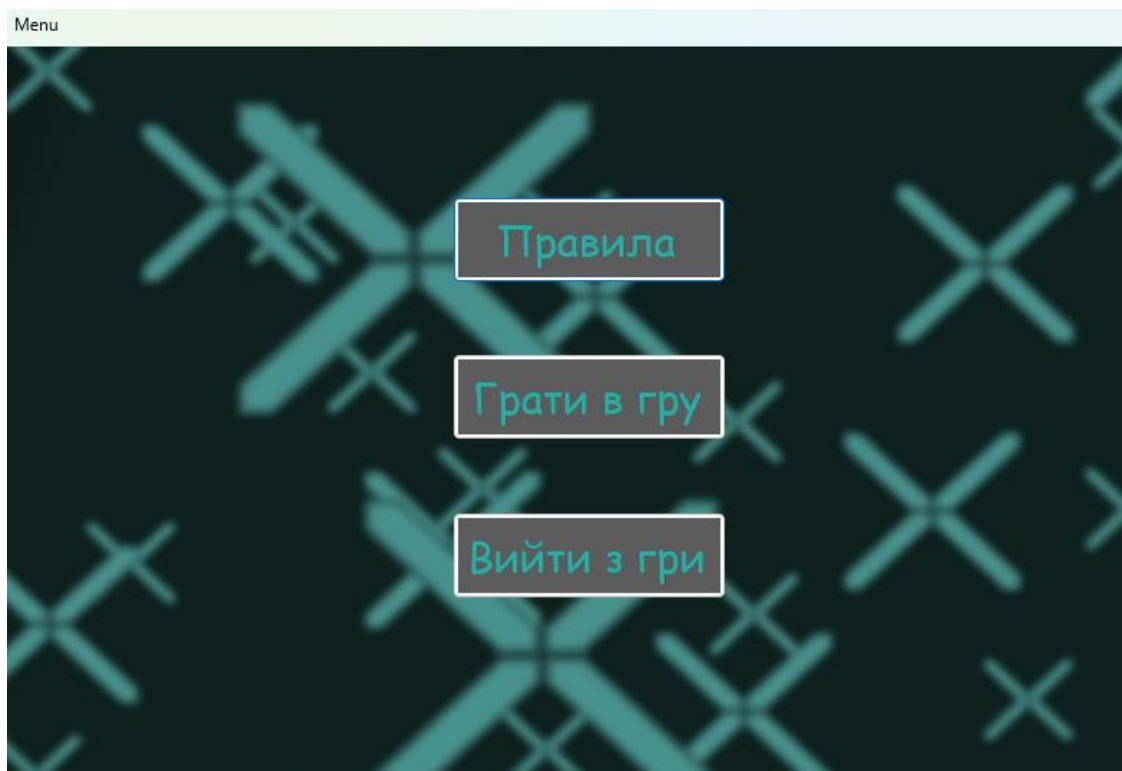


Рисунок 2.18 – Головне меню програмного продукту

При натисканні на кнопку меню “Правила” буде відкрито нове вікно Rules, а минуле приховано (рисунок 2.19).

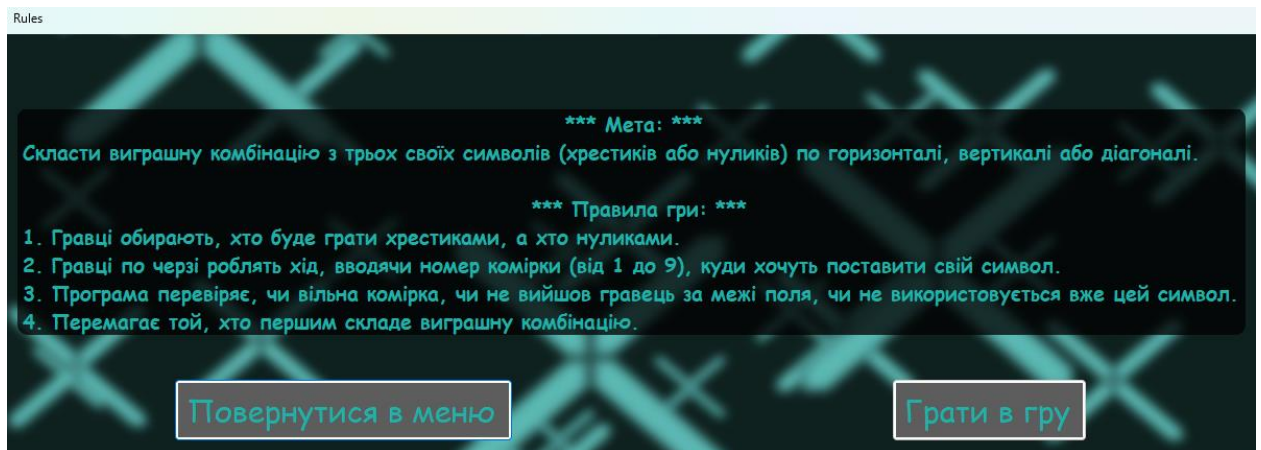


Рисунок 2.19 – Вікно з правилами

Після того, як гравці ознайомилися з правилами можуть повернутися в меню або запустити гру. Якщо гравці обрали запустити гру, відкривається нове вікно Game, а минуле вікно приховується (рисунок 2.20).

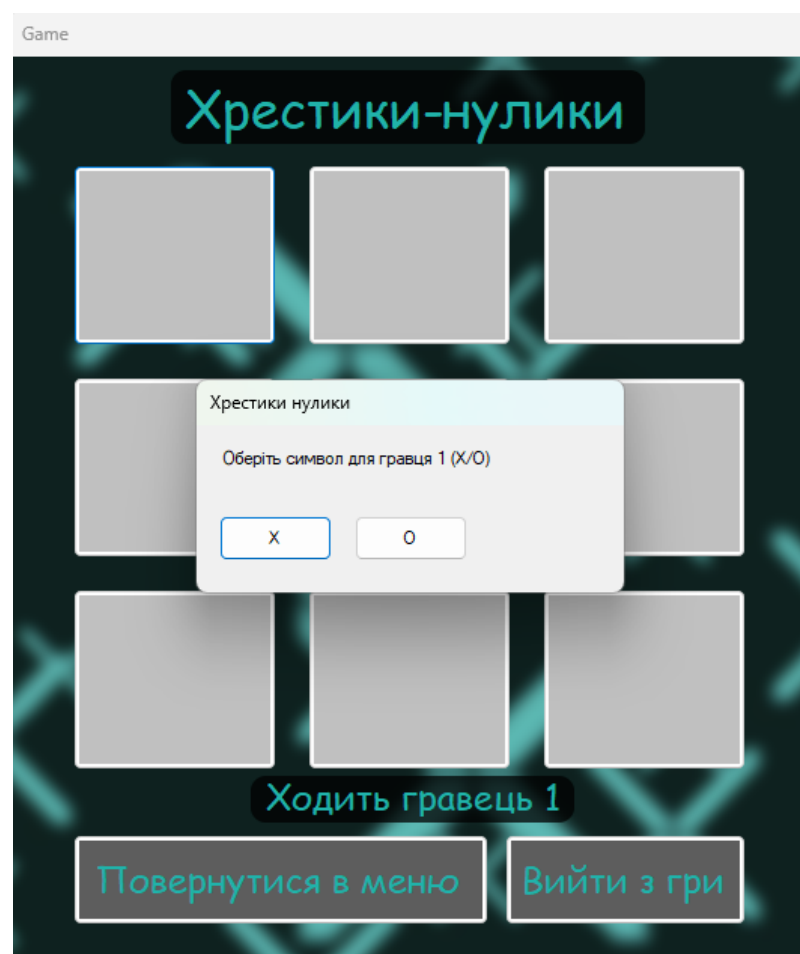


Рисунок 2.20 – Запуск гри

Після того, як гравці обрали запуск гри відкривається ще одне вікно де просять вибрати символ для гравця 1. Коли обрано символ для гравця 1, гра автоматично призначає протилежний символ для гравця 2 в залежності від символу гравця 1.

Гравці можуть натискати на кнопки гральної дошки після вибору символів (рисунок 2.21).

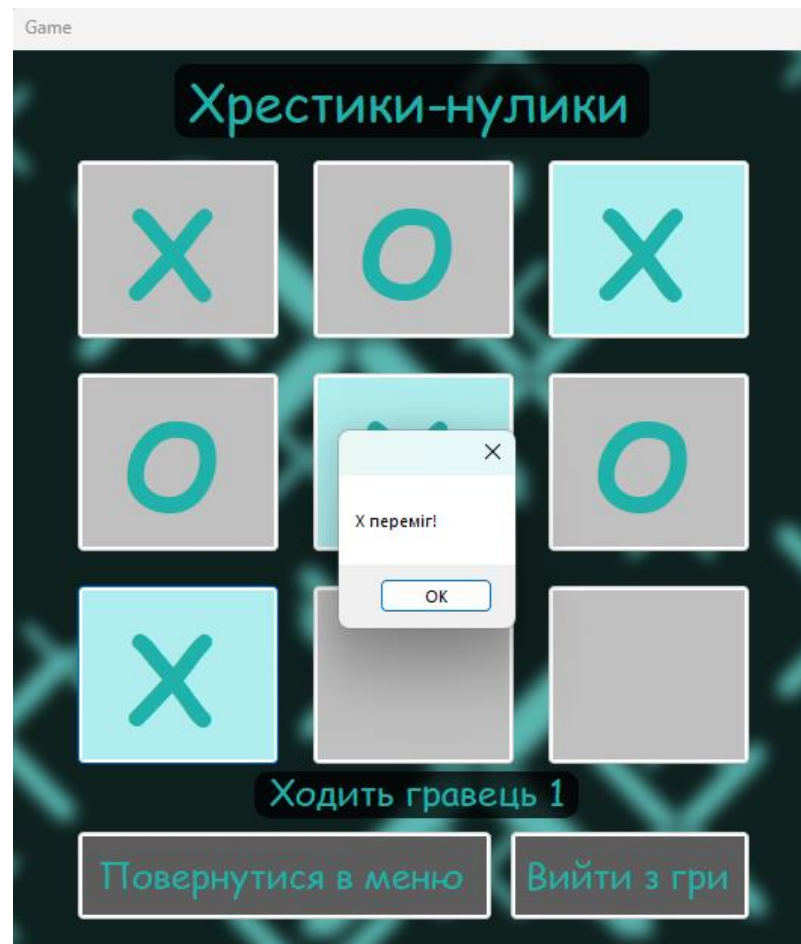


Рисунок 2.21 – Введені ходи гравців

Якщо є виграшна комбінація або нічия виводиться повідомлення про перемогу або нічию та запитується чи хочуть гравці продовжити гру (рисунок 2.22).

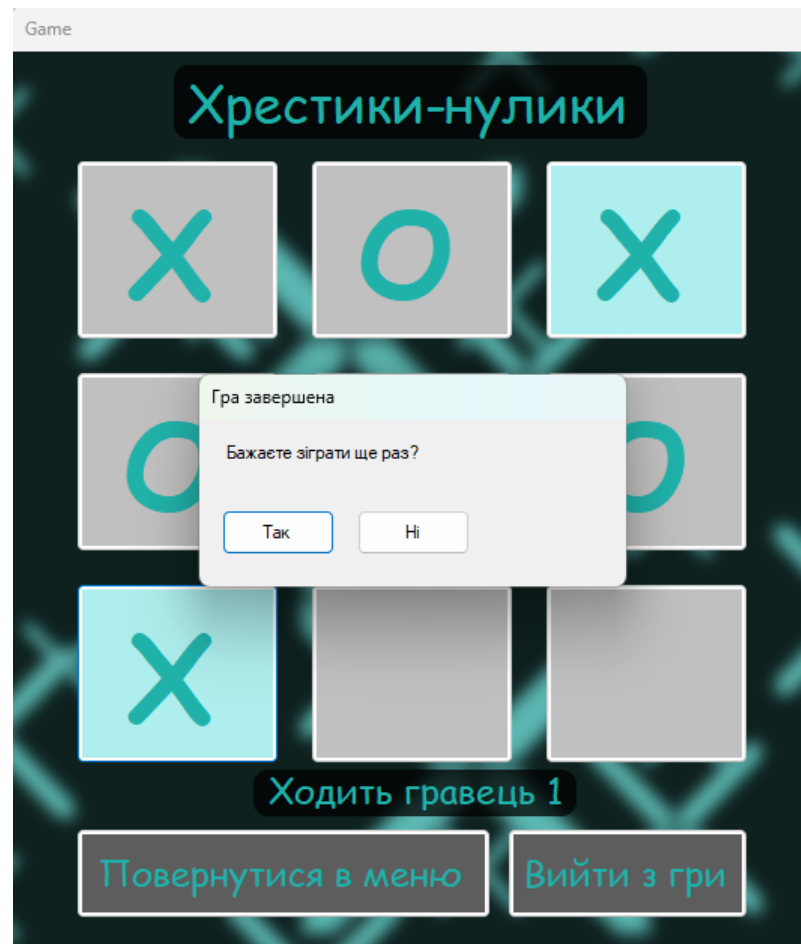


Рисунок 2.22 – Запит про продовження гри

Якщо гравці вибирають “Так”, то гральна дошка очищається та першим будемо ходити гравець протилежного ходу (рисунок 2.23).

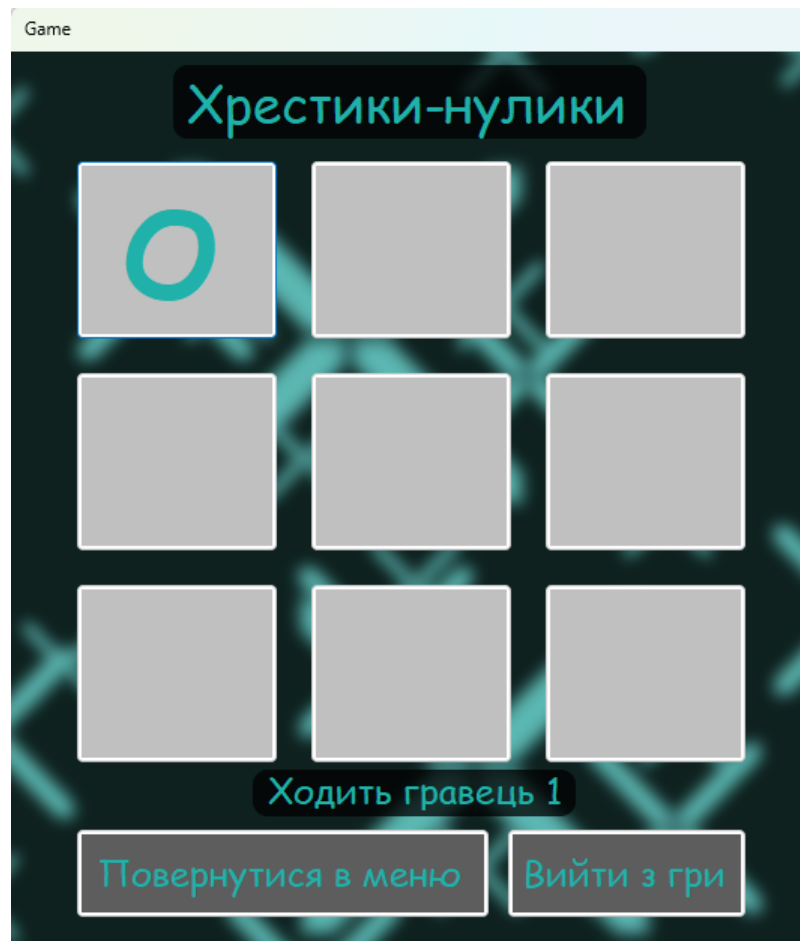


Рисунок 2.23 – Перезапуск гри

Якщо гравці вибирають “Ні”, то гру буде завершено.

При натисканні на кнопку меню “Вийти з гри”, то гру буде завершено.

ВИСНОВКИ

У рамках курсової роботи переді мною була поставлена задача розробити гру “Хрестики-нулики” з консольним та графічним інтерфейсами. Для цього я використав мови програмування C++ та C#, а також технологію WinForms для створення графічного інтерфейсу.

Протягом роботи над проєктом я розробив алгоритм гри, враховуючи правила “Хрестиків-нуликів”, створив прототипи консольного та графічного інтерфейсів, написав програмний код та реалізував логіку гри, включаючи перевірку виграшних комбінацій та завершення гри.

Результатом моєї роботи стала гра “Хрестики-нулики”, яка дозволяє користувачам грати в цю класичну гру як в консольному, так і в графічному інтерфейсах. Гра має зручний та інтуїтивно зрозумілий інтерфейс, а також ефективний алгоритм для обробки ходів гравців та визначення переможця.

Застосування гри “Хрестики-нулики” можна знайти в різних контекстах, як:

- гра може бути використана для простого відпочинку та розваги;
- гра може бути використана для проведення змагань між гравцями.

В процесі роботи над проєктом я оволодів навичками розробки програмного забезпечення з використанням мов програмування C++ та C#, а також технології WinForms. Також я вдосконалив навички написання коду, алгоритмізації, тестування та відлагодження програм.

Я планую продовжити роботу над проєктом, додавши нові функції, такі як можливість грати з комп'ютером, зміна розмірову ігрового поля, статистика ігор та інше. Це дозволить зробити гру ще більш цікавою та привабливою для користувачів.

СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

1. Грицюк Ю. І., Рак Т. Є. Програмування мовою C++: навч. посіб. / за ред. Дудка В.В. Львів: ЛДУ БЖД, 2011. 292 с.
2. Глинський Я. М., Анохін В. Є., Ряжська В. А. C++ і C++ Builder: навч. посіб. Львів: Деол, СПД Глинський, 2003. 192 с.
3. Васильєв О. М. Програмування на C++ в прикладах і задачах: навч. посіб. Київ: Ліра-К, 2019. 382 с.
4. Трифomenко О. Г., Прокоп Ю. В., Швайко І. Г., Буката Л. В. C++. Основи програмування. Теорія та практика / за ред. Трифomenка О. Г. Одеса: Фенікс, 2010. 544 с.
5. Щедріна О. І. Алгоритмізація та програмування процедур обробки інформації C++: навч. посіб. Київ: КНЕУ, 2001. 244 с.
6. Зеленський О. С., Лисенко В. С. Розробка Windows-додатків на мові C#: навч. посіб. Кривий Ріг: Державний університет економіки і технологій, 2023. 357 с.
7. МЕТОДИЧНІ ВКАЗІВКИ ДО ВИКОНАННЯ КУРСОВИХ РОБІТ ДЛЯ ЗДОБУВАЧІВ ОСВІТИ СПЕЦІАЛЬНОСТІ 121 «ІНЖЕНЕРІЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ»: МЕТОДИЧНІ ВКАЗІВКИ / Відокремлений структурний підрозділ «Полтавський політехнічний фаховий коледж Національного технічного університету «Харківський політехнічний інститут»; [уклад.: В.В.Олійник]. – Полтава: ВСП ППФК НТУ «ХПІ», 2022. – 22 с.

ДОДАТОК А. ВИХІДНІ КОДИ

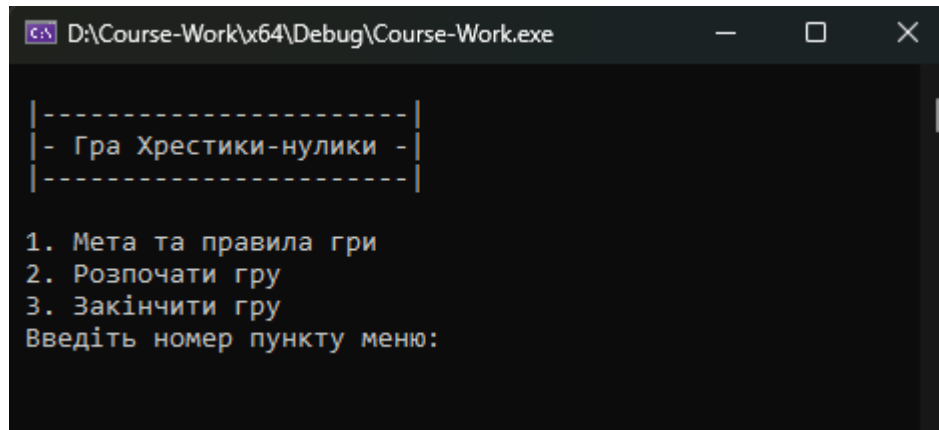


Додаток А.1 – Посилання на репозиторій консольного варіанту



Додаток А.2 – Посилання на репозиторій графічного варіанту

ДОДАТОК Б. РЕЗУЛЬТАТИ ТЕСТУВАННЯ



```

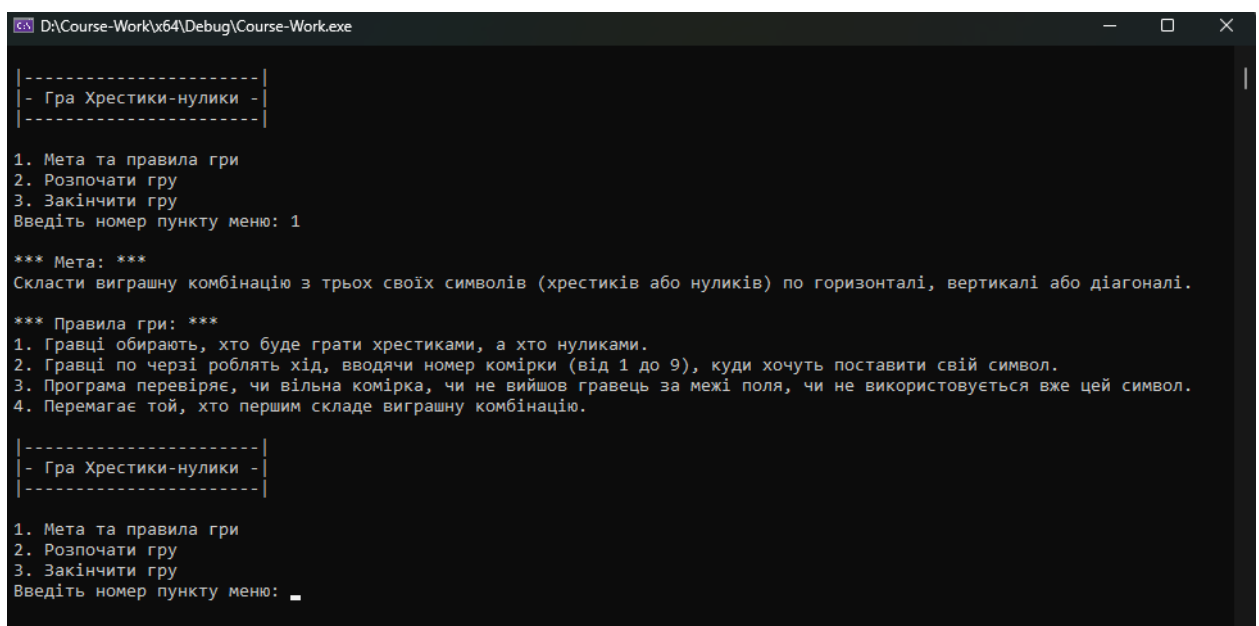
D:\Course-Work\64\Debug\Course-Work.exe

|-----|
| - Гра Хрестики-нулики - |
|-----|

1. Мета та правила гри
2. Розпочати гру
3. Закінчити гру
Введіть номер пункту меню:

```

Додаток Б.1 – Запуск програми



```

D:\Course-Work\64\Debug\Course-Work.exe

|-----|
| - Гра Хрестики-нулики - |
|-----|

1. Мета та правила гри
2. Розпочати гру
3. Закінчити гру
Введіть номер пункту меню: 1

*** Мета: ***
Скласти виграшну комбінацію з трьох своїх символів (хрестиків або нуликів) по горизонталі, вертикалі або діагоналі.

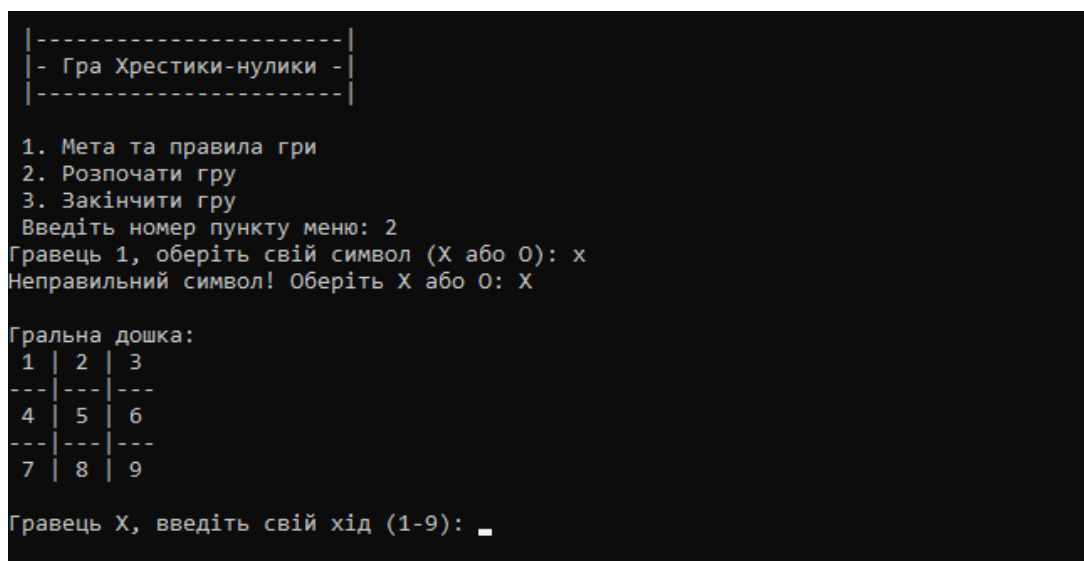
*** Правила гри: ***
1. Гравці обирають, хто буде грати хрестиками, а хто нуликами.
2. Гравці по черзі роблять хід, вводючи номер комірки (від 1 до 9), куди хочуть поставити свій символ.
3. Програма перевіряє, чи вільна комірка, чи не вийшов гравець за межі поля, чи не використовується вже цей символ.
4. Перемагає той, хто першим складе виграшну комбінацію.

|-----|
| - Гра Хрестики-нулики - |
|-----|

1. Мета та правила гри
2. Розпочати гру
3. Закінчити гру
Введіть номер пункту меню: 1

```

Додаток Б.2 – Перевірка пункту “1”



```

|-----|
| - Гра Хрестики-нулики - |
|-----|

1. Мета та правила гри
2. Розпочати гру
3. Закінчити гру
Введіть номер пункту меню: 2
Гравець 1, оберіть свій символ (X або O): x
Неправильний символ! Оберіть X або O: X

Гральна дошка:
1 | 2 | 3
---|---|---
4 | 5 | 6
---|---|---
7 | 8 | 9

Гравець X, введіть свій хід (1-9):

```

Додаток Б.3 – Перевірка на коректність введенного символу

```

Гральна дошка:
1 | 2 | 3
---|---|---
4 | 5 | 6
---|---|---
7 | 8 | 9

Гравець X, введіть свій хід (1-9): 0
Неправильний хід, спробуйте ще раз.
Гравець X, введіть свій хід (1-9): 1
X |  | 
---|---|---
  |  | 
---|---|---
  |  | 

Гравець O, введіть свій хід (1-9): 2
X | O | 
---|---|---
  |  | 
---|---|---
  |  | 

Гравець X, введіть свій хід (1-9): 3
X | O | X
---|---|---
  |  | 
---|---|---
  |  | 

Гравець O, введіть свій хід (1-9): 10
Неправильний хід, спробуйте ще раз.
Гравець O, введіть свій хід (1-9):

```

Додаток Б.4 – Перевірка на коректність введення ходів

```

X | X | X
---|---|---
O | O | 
---|---|---
  |  | 
---|---|---
  |  | 

Гравець X переміг!
Грати ще раз? (Так/Ні):

```

Додаток Б.5 – Перевірка на виграшну комбінацію по горизонталі

```

O | X | 
---|---|---
O | X | 
---|---|---
O |  | 
---|---|---
  |  | 

Гравець O переміг!
Грати ще раз? (Так/Ні):

```

Додаток Б.6 – Перевірка на виграшну комбінацію по вертикалі

```

X | O | 
---|---|---
  | X | O
---|---|---
  |  | X
---|---|---
  |  | 

Гравець X переміг!
Грати ще раз? (Так/Ні):

```

Додаток Б.7 – Перевірка на виграшну комбінацію по діагоналі

```

O | X | O
---|---|---
O | X | O
---|---|---
X | O | X
Нічия!
Грати ще раз? (Так/Ні):

```

Додаток Б.8 – Перевірка на нічию

```

|-----|
| - Гра Хрестики-нулики - |
|-----|

1. Мета та правила гри
2. Розпочати гру
3. Закінчити гру
Введіть номер пункту меню: 4
Некоректний вибір! Спробуйте ще раз.

|-----|
| - Гра Хрестики-нулики - |
|-----|

1. Мета та правила гри
2. Розпочати гру
3. Закінчити гру
Введіть номер пункту меню:

```

Додаток Б.9 – Перевірка на неправильний введений пункт

```

|-----|
| - Гра Хрестики-нулики - |
|-----|

1. Мета та правила гри
2. Розпочати гру
3. Закінчити гру
Введіть номер пункту меню: 2
Гравець 1, оберіть свій символ (X або O): X

Гральна дошка:
1 | 2 | 3
---|---|---
4 | 5 | 6
---|---|---
7 | 8 | 9

Гравець X, введіть свій хід (1-9):

```

Додаток Б.10 – Гральна дошка при умові, що show_numbers = true


```

|-----|
| - Гра Хрестики-нулики - |
|-----|

1. Мета та правила гри
2. Розпочати гру
3. Закінчити гру
Введіть номер пункту меню: 2
Гравець 1, оберіть свій символ (X або O): X

Гральна дошка:
|   |   |
---|---|---
|   |   |
---|---|---
|   |   |

Гравець X, введіть свій хід (1-9):

```

Додаток Б.11 – Гральна дошка при умові, що show_numbers = false

```

|-----|
| - Гра Хрестики-нулики - |
|-----|

1. Мета та правила гри
2. Розпочати гру
3. Закінчити гру
Введіть номер пункту меню: 1

*** Мета: ***
Скласти виграшну комбінацію з трьох своїх символів (хрестиків або нуликів) по горизонталі, вертикалі або діагоналі.

*** Правила гри: ***
1. Гравці обирають, хто буде грати хрестиками, а хто нуликами.
2. Гравці по черзі роблять хід, вводячи номер комірки (від 1 до 9), куди хочуть поставити свій символ.
3. Програма перевіряє, чи вільна комірка, чи не вийшов гравець за межі поля, чи не використовується вже цей символ.
4. Перемагає той, хто першим складе виграшну комбінацію.

|-----|
| - Гра Хрестики-нулики - |
|-----|

1. Мета та правила гри
2. Розпочати гру
3. Закінчити гру
Введіть номер пункту меню: 2
Гравець 1, оберіть свій символ (X або O):

```

Додаток Б.12 – Перевірка на правильність вводу пунктів з меню

```

|-----|
| - Гра Хрестики-нулики - |
|-----|

1. Мета та правила гри
2. Розпочати гру
3. Закінчити гру
Введіть номер пункту меню: 2
Гравець 1, оберіть свій символ (X або O): X

Гральна дошка:
1 | 2 | 3
---|---|---
4 | 5 | 6
---|---|---
7 | 8 | 9

Гравець X, введіть свій хід (1-9): 1
X |   |
---|---|---
   |   |
---|---|---
   |   |

Гравець O, введіть свій хід (1-9): 2
X | O |
---|---|---
   |   |
---|---|---
   |   |

Гравець X, введіть свій хід (1-9): 3
X | O | X
---|---|---
   |   |
---|---|---
   |   |

Гравець O, введіть свій хід (1-9): 4
X | O | X
---|---|---
O |   |
---|---|---
   |   |

Гравець X, введіть свій хід (1-9):

```

Додаток Б.13 – Перевірка обробки ходів

```

X | X | X
---|---|---
O | O |
---|---|---
   |   |
Гравець X переміг!
Грати ще раз? (Так/Ні):

```

Додаток Б.14 – Перевірка виграшної комбінації по горизонталі

```

O | X |
---|---|---
O | X |
---|---|---
O |   |
Гравець O переміг!
Грати ще раз? (Так/Ні):

```

Додаток Б.15 – Перевірка виграшної комбінації по вертикалі

```

0 | X |
---|---|---
  | O | X
---|---|---
  |   | O
Гравець O переміг!
Грати ще раз? (Так/Ні):

```

Додаток Б.16 – Перевірка виграшної комбінації по діагоналі

```

X | O | X
---|---|---
X | O | X
---|---|---
O | X | O
Нічия!
Грати ще раз? (Так/Ні): _

```

Додаток Б.17 – Перевірка на нічию

```

Гравець O, введіть свій хід (1-9): 4
0 | X | O
---|---|---
O | X | O
---|---|---
X | O | X
Нічия!
Грати ще раз? (Так/Ні): Так
Гравець X, введіть свій хід (1-9): 1
X |   |
---|---|---
  |   |
---|---|---
  |   |
Гравець O, введіть свій хід (1-9):

```

Додаток Б.18 – Перевірка очищення дошки

The screenshot shows two windows. The top window is 'GoogleTest_Console' displaying C++ code for a Tic Tac Toe game. The code includes headers, a custom output redirection class, and a test function. The bottom window is 'Консоль отладки Microsoft Visual Studio' showing the test results for 'MenuTest.OutputTest', which passed.

```

1  #include "pch.h"
2  #include "gtest/gtest.h"
3  #include <iostream>
4  #include <Windows.h>
5
6  using namespace std;
7
8  class CoutRedirect {
9  public:
10     CoutRedirect(std::streambuf* new_buffer) : old(std::cout.rdbuf(new_buffer)) {}
11     ~CoutRedirect() {
12         std::cout.rdbuf(old);
13     }
14 private:
15     std::streambuf* old;
16 };
17
18 TEST(MenuTest, OutputTest) {
19     // Очікуваний результат
20     std::string expected_output =
21         "  |   |   |\n"
22         "  |   |   |\n"
23         "  |   |   |\n"
24         "1. Має та правила гри\n"
25         "2. Розпочати гру\n"
26         "3. Закінчити гру\n";
27
28     std::stringstream buffer;
29     CoutRedirect redirect(buffer.rdbuf());
30
31     menu();
32
33     // Порівняємо вивід з очікуваним результатом
34     std::string output = buffer.str();
35     EXPECT_EQ(output, expected_output);
36 }
37
38 int main(int argc, char** argv) {
39     ::testing::InitGoogleTest(&argc, argv);
40     return RUN_ALL_TESTS();
41 }

```

```

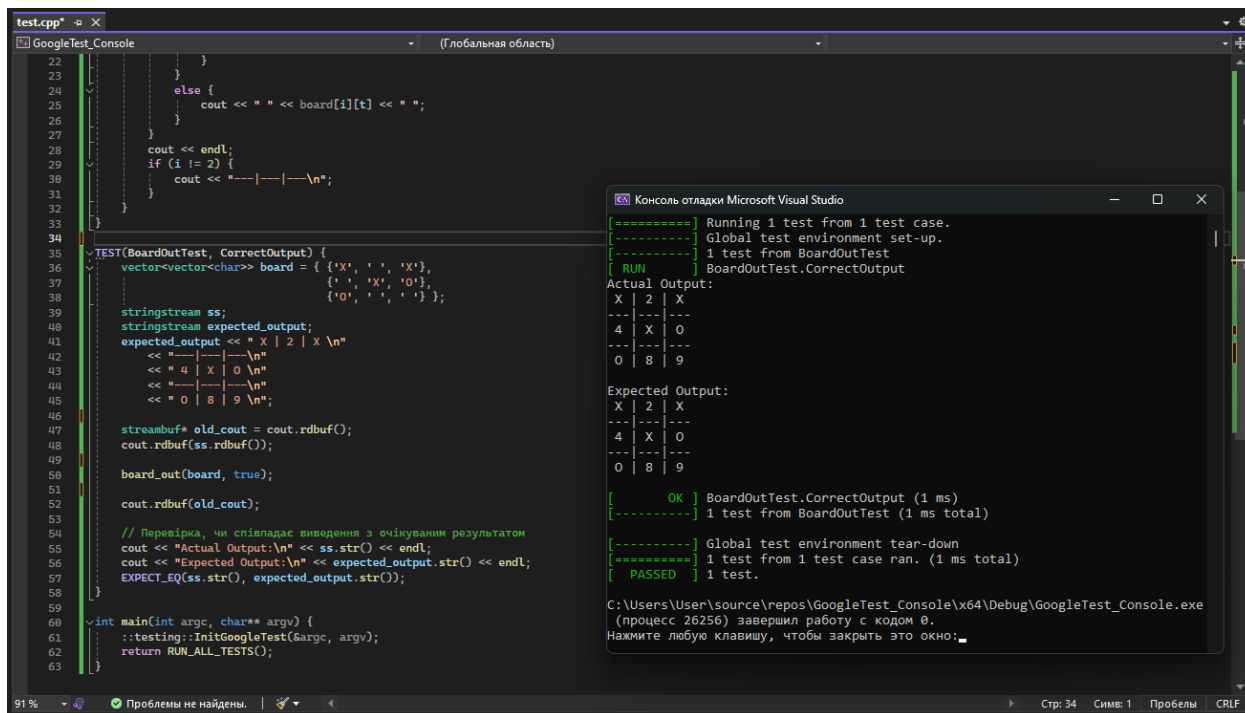
===== Running 1 test from 1 test case.
----- Global test environment set-up.
----- 1 test from MenuTest
RUN      MenuTest.OutputTest
[ OK ]   MenuTest.OutputTest (0 ms)
----- 1 test from MenuTest (0 ms total)

----- Global test environment tear-down
----- 1 test from 1 test case ran. (1 ms total)
[PASSED] 1 test.

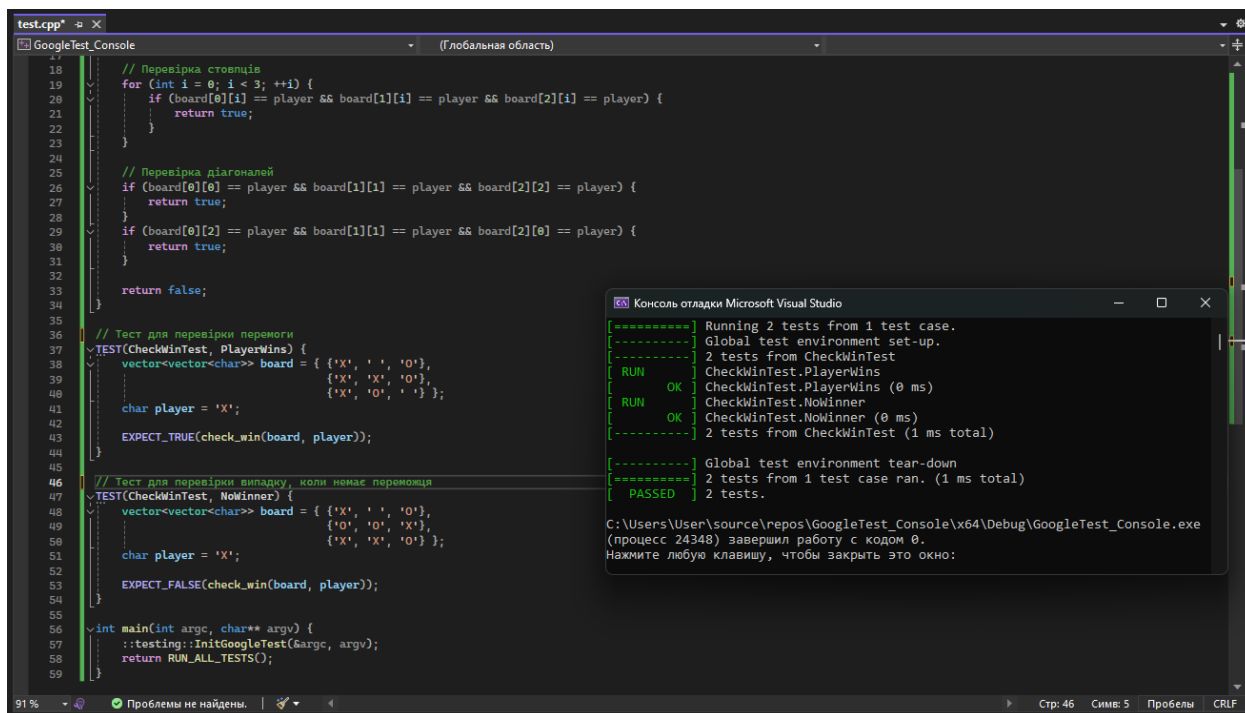
C:\Users\User\source\repos\GoogleTest_Console\x64\Debug\GoogleTest_Console.exe (процесс
27372) завершил работу с кодом 0.
Нажмите любую клавишу, чтобы закрыть это окно...

```

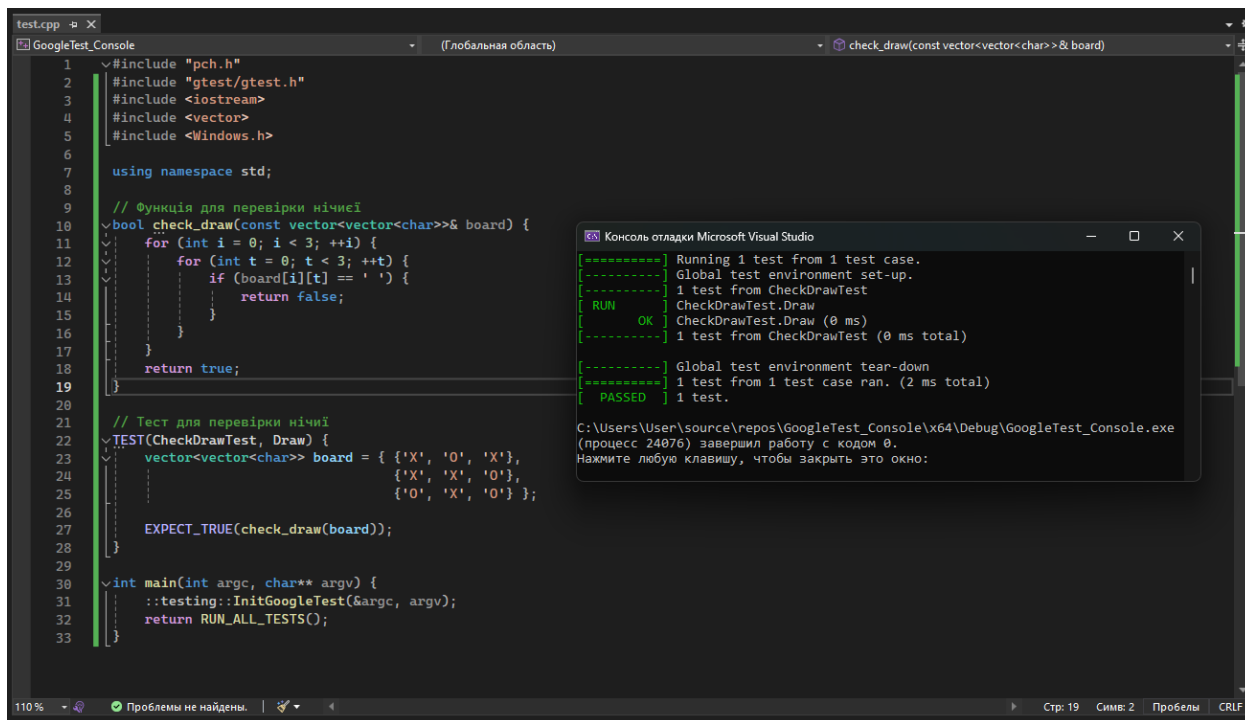
Додаток Б.19 – Тестування на правильність виведення меню



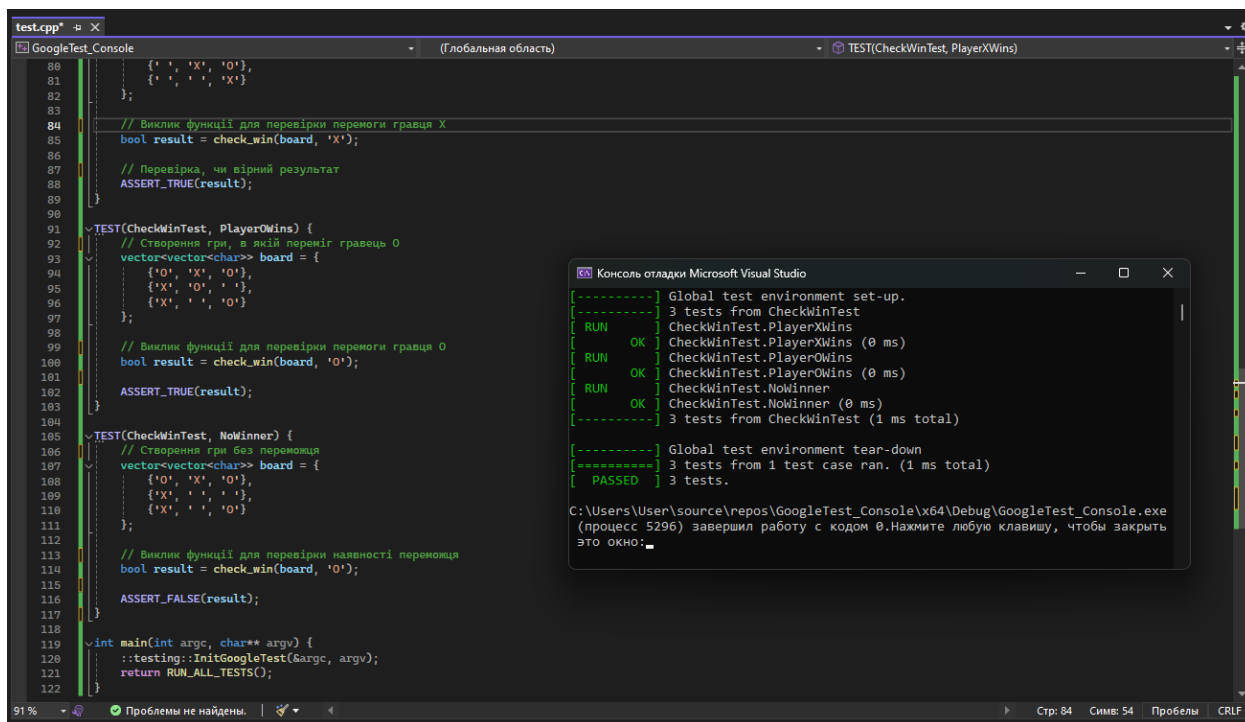
Додаток Б.20 – Тестування на правильність виведення гральної дошки



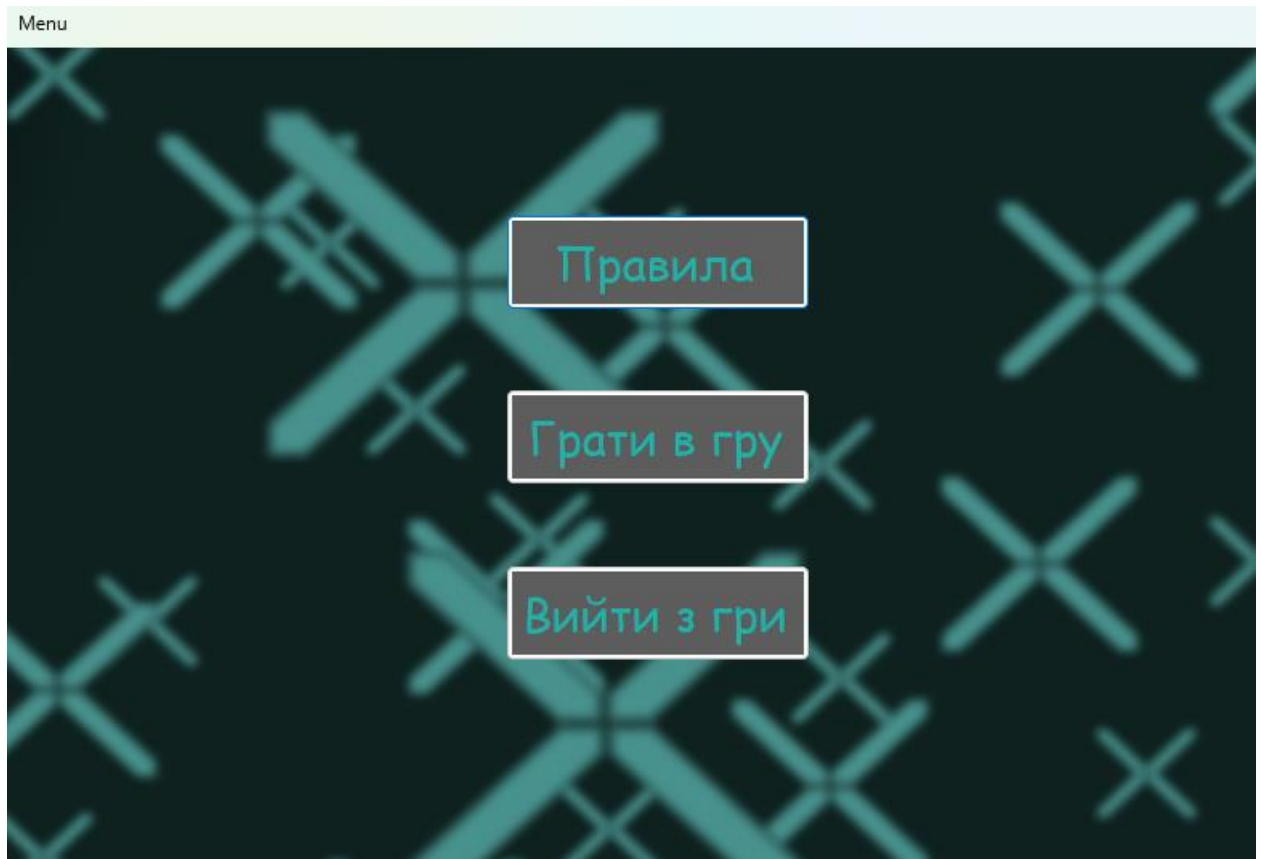
Додаток Б.21 – Тестування на правильність функції для перевірки перемоги



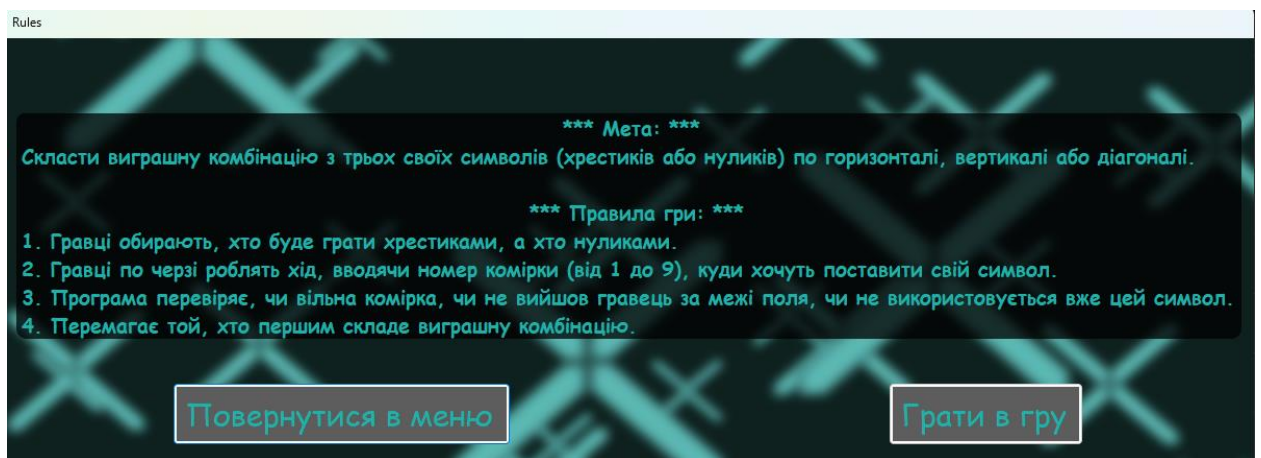
Додаток Б.22 – Тестування на правильність функції для перевірки нічиї



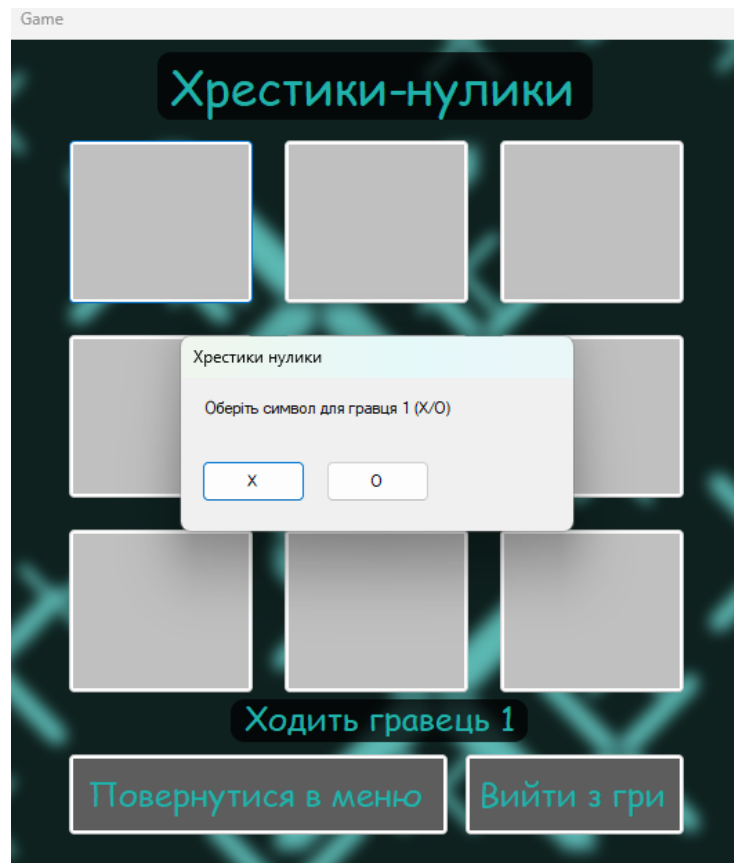
Додаток Б.23 – Тестування самої гри



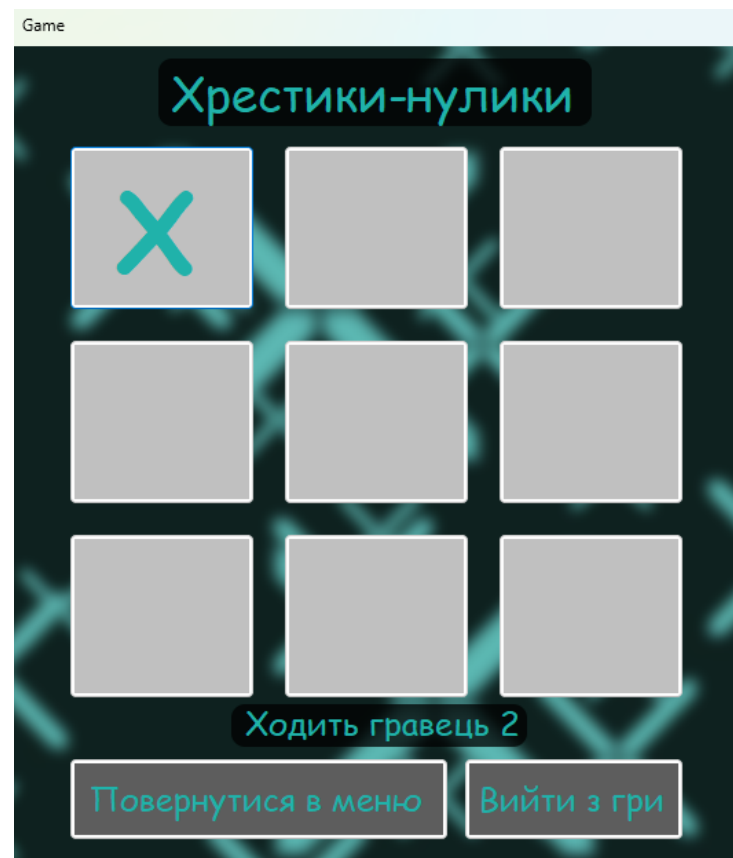
Додаток Б.24 – Запуск програми



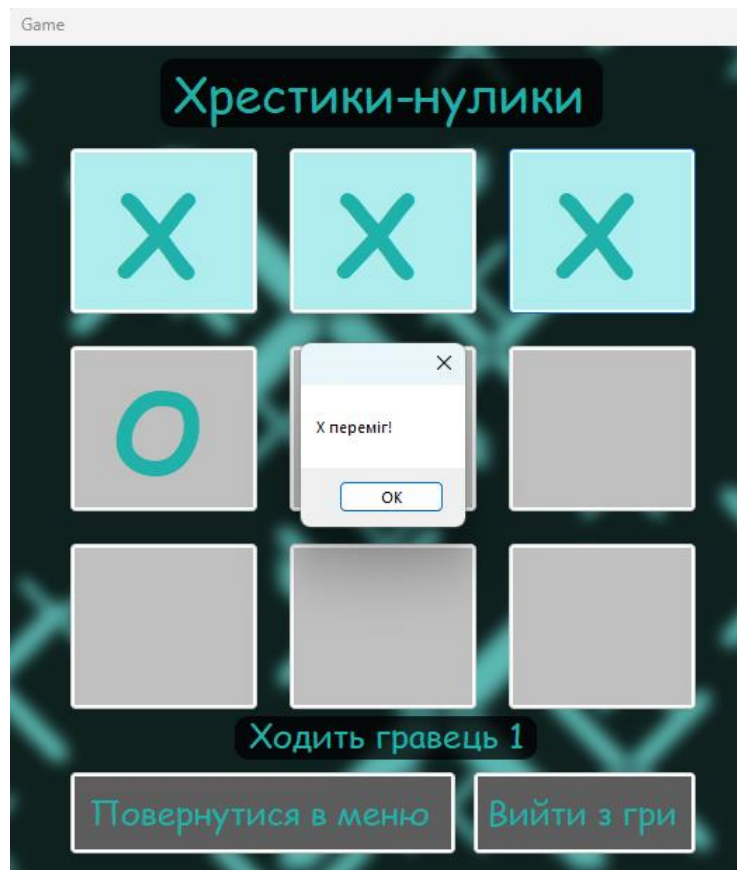
Додаток Б.25 – Перевірка пункту “Правила”



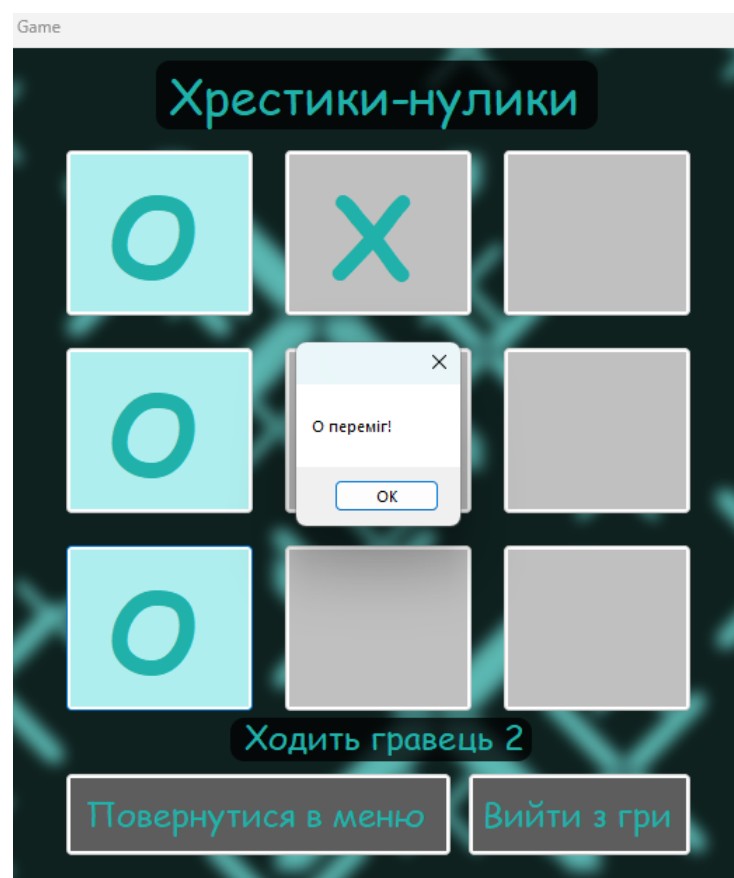
Додаток Б.26 – Перевірка пункту “Грати в гру” та на відображення вікна для вибору символів



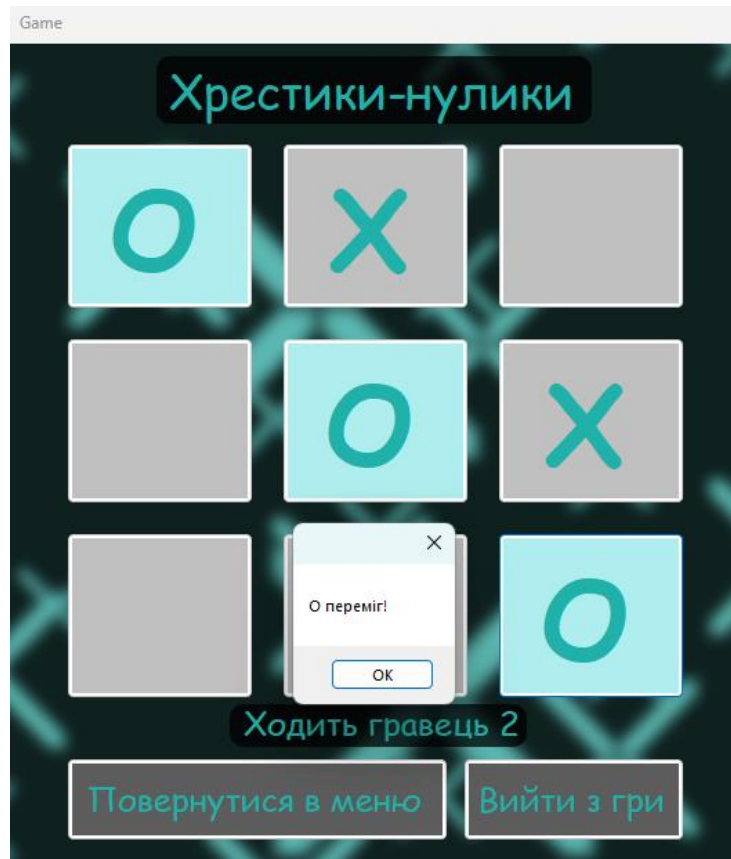
Додаток Б.27 – Перевірка чи коректно працює вікно вибору символів



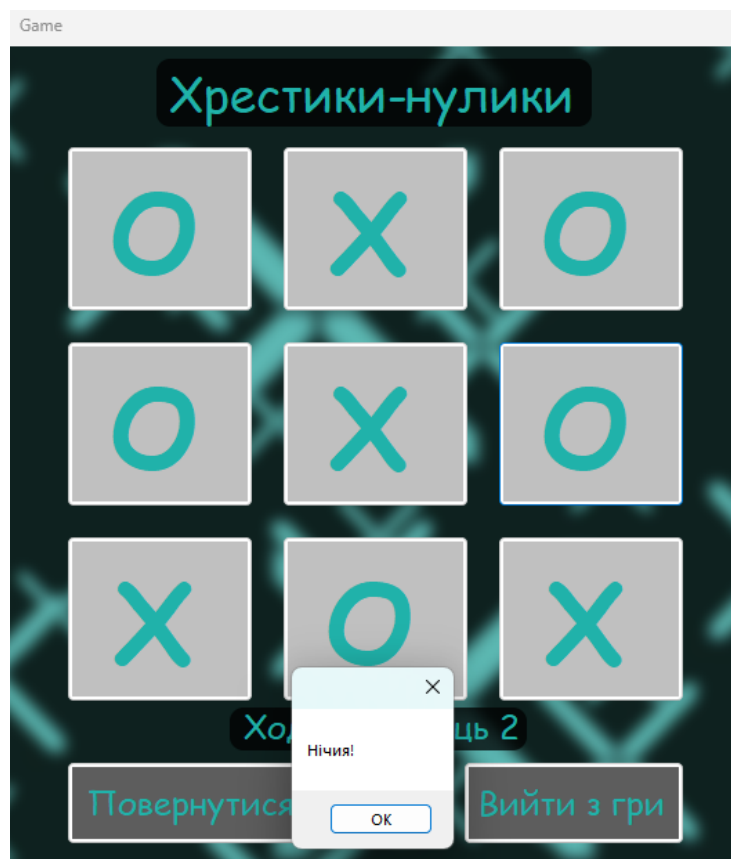
Додаток Б.28 – Перевірка виграшної комбінації по горизонталі



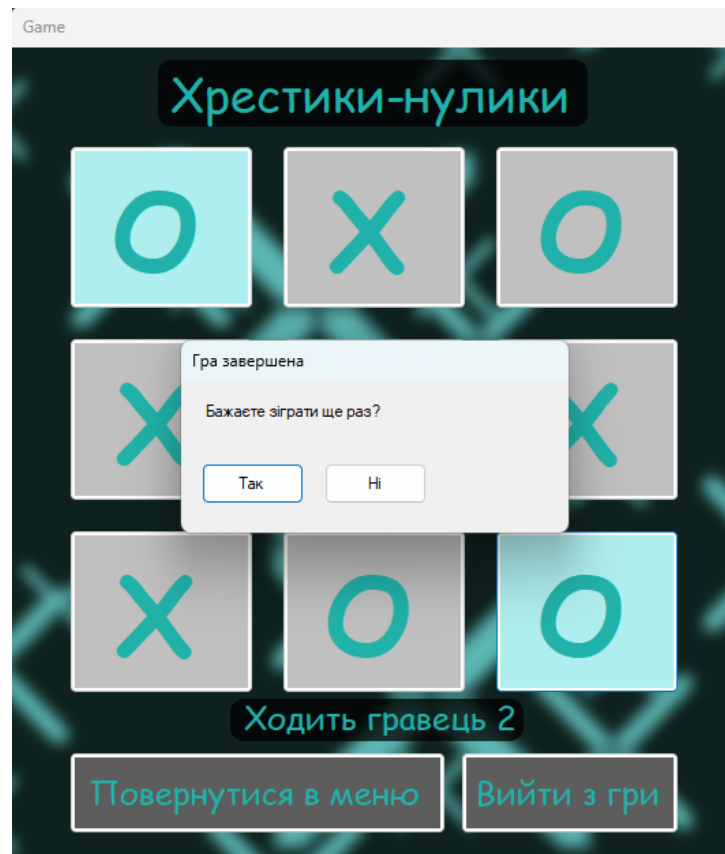
Додаток Б.29 – Перевірка виграшної комбінації по вертикалі



Додаток Б.30 – Перевірка виграшної комбінації по діагоналі



Додаток Б.31 – Перевірка на нічию



Додаток Б.32 – Перевірка чи з'являється повідомлення із запитом про продовження гри

```
namespace Course_Work_Graphics
{
    Ссылка: 8 | User: 3 дн. назад | Автор: 1, изменение: 1
    public partial class Menu : Form
    {
        // Ініціалізація компонентів форми Menu
        Ссылка: 3 | User: 3 дн. назад | Автор: 1, изменение: 1
        public Menu()
        {
            InitializeComponent();
            this.FormClosing += Menu_FormClosing; // Закриття форми
        }

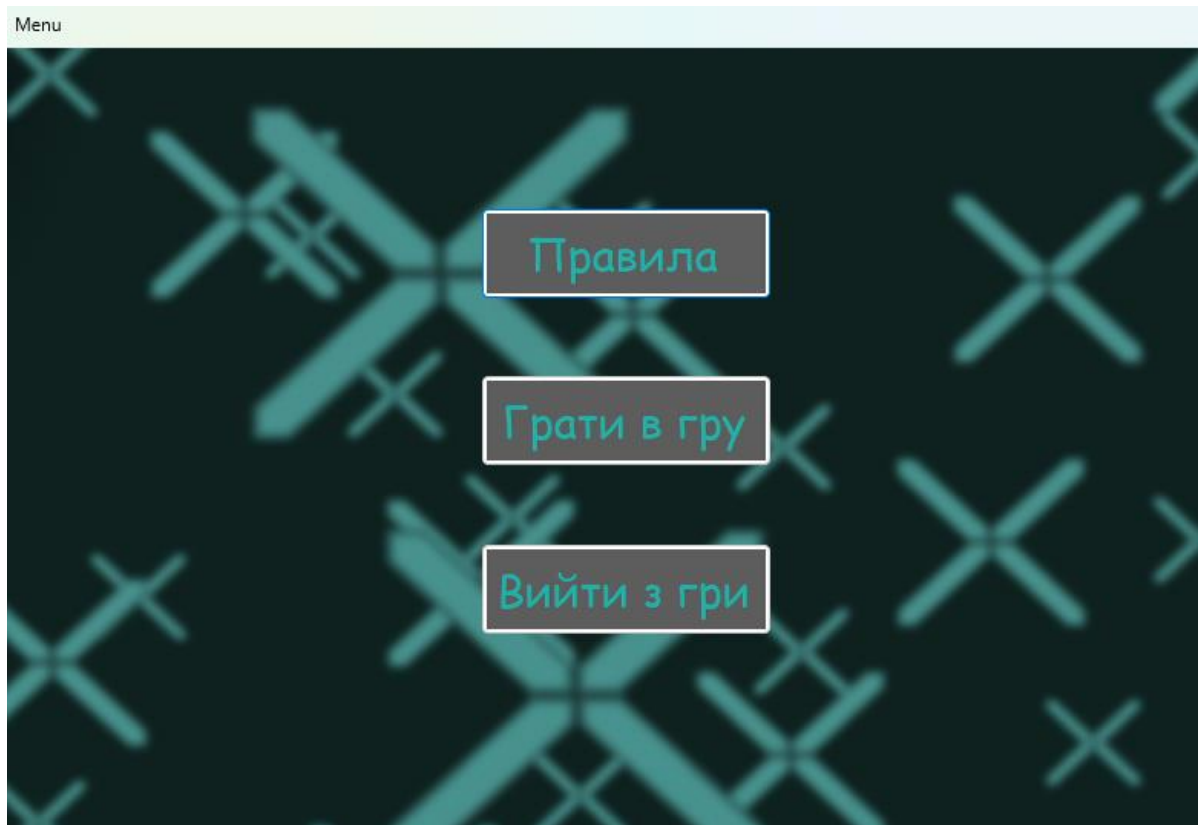
        Ссылка: 1 | User: 3 дн. назад | Автор: 1, изменение: 1
        private void ButRules_Click(object sender, EventArgs e)
        {
            Rules RulesForm = new Rules(); // Створюємо екземпляр форми Rules
            RulesForm.Show(); // Відображаємо форму Rules
            this.Hide(); // Приховуємо поточну форму Menu
        }

        Ссылка: 1 | User: 3 дн. назад | Автор: 1, изменение: 1
        private void ButClose_Click(object sender, EventArgs e)
        {
            Application.Exit(); // Завершення роботи
        }

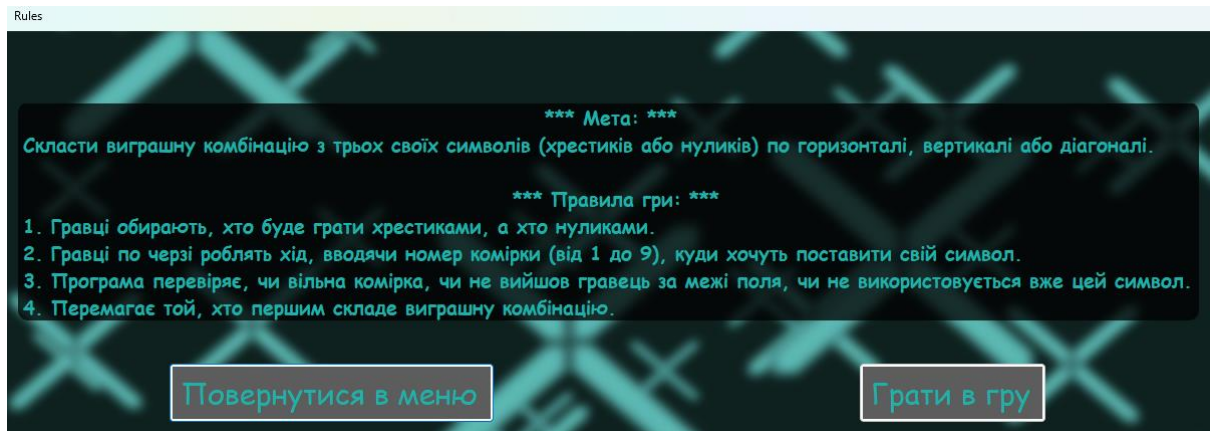
        Ссылка: 1 | User: 3 дн. назад | Автор: 1, изменение: 1
        private void Menu_FormClosing(object sender, FormClosingEventArgs e)
        {
            if (e.CloseReason == CloseReason.UserClosing)
            {
                Application.Exit(); // Завершення роботи
            }
        }

        Ссылка: 1 | User: 3 дн. назад | Автор: 1, изменение: 1
        private void ButGame_Click(object sender, EventArgs e)
        {
            Game GameForm = new Game(); // Створюємо екземпляр форми Game
            GameForm.Show(); // Відображаємо форму Game
            this.Hide(); // Приховуємо поточну форму Menu
        }
    }
}
```

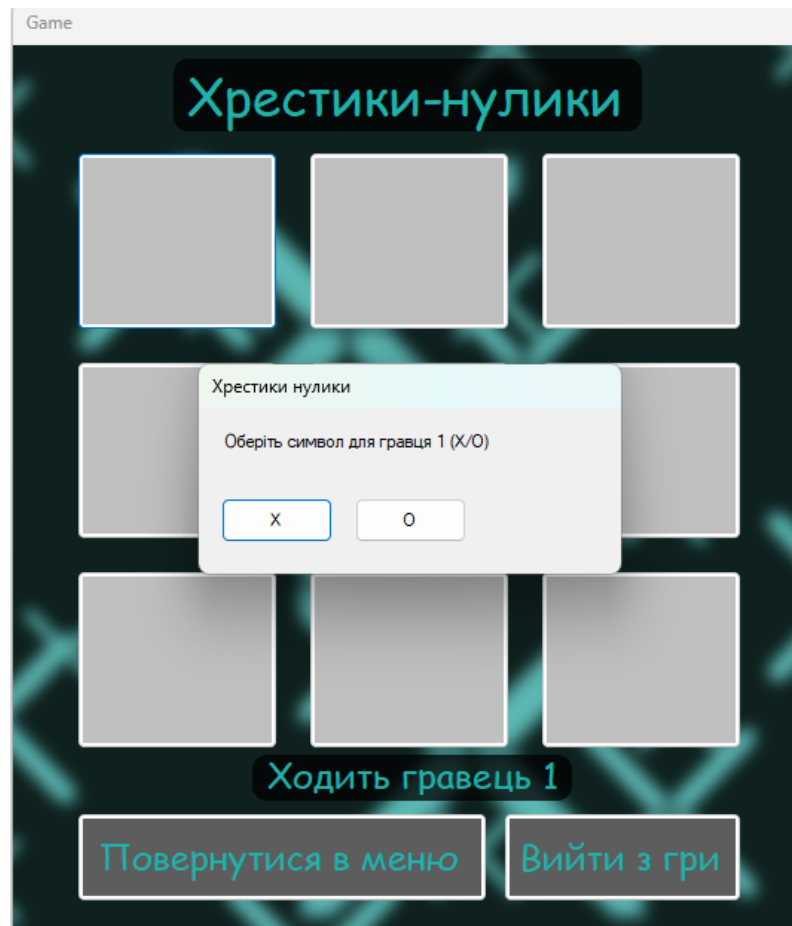
Додаток Б.33 – Ознайомлення з кодом форми меню



Додаток Б.34 – Запуск форми меню



Додаток Б.35 – Перехід у форму правила за допомогою кнопки в меню



Додаток Б.36 – Перехід у форму гри за допомогою кнопки в меню

```

namespace Course_Work_Graphics
{
    public partial class Rules : Form
    {
        // Ініціалізація компонентів форми Rules
        public Rules()
        {
            InitializeComponent();
            this.FormClosing += Rules_FormClosing; // Закриття форми
        }

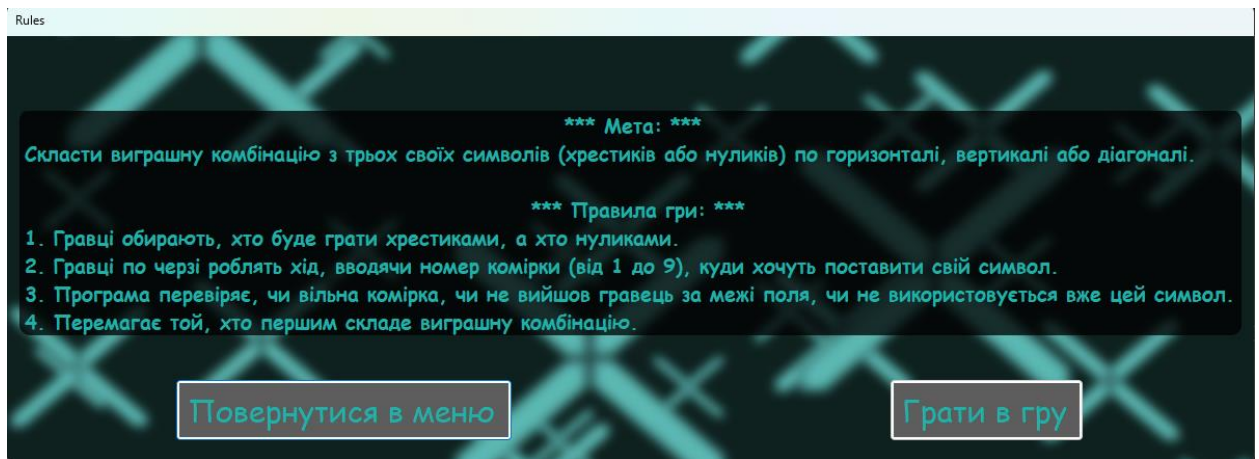
        private void ButBackToMenu_Click(object sender, EventArgs e)
        {
            Menu MenuForm = new Menu(); // Створюємо екземпляр форми Menu
            MenuForm.Show(); // Відображаємо форму Menu
            this.Hide(); // Приховуємо поточну форму Rules
        }

        private void Rules_FormClosing(object sender, FormClosingEventArgs e)
        {
            if (e.CloseReason == CloseReason.UserClosing)
            {
                Application.Exit(); // Завершення роботи
            }
        }

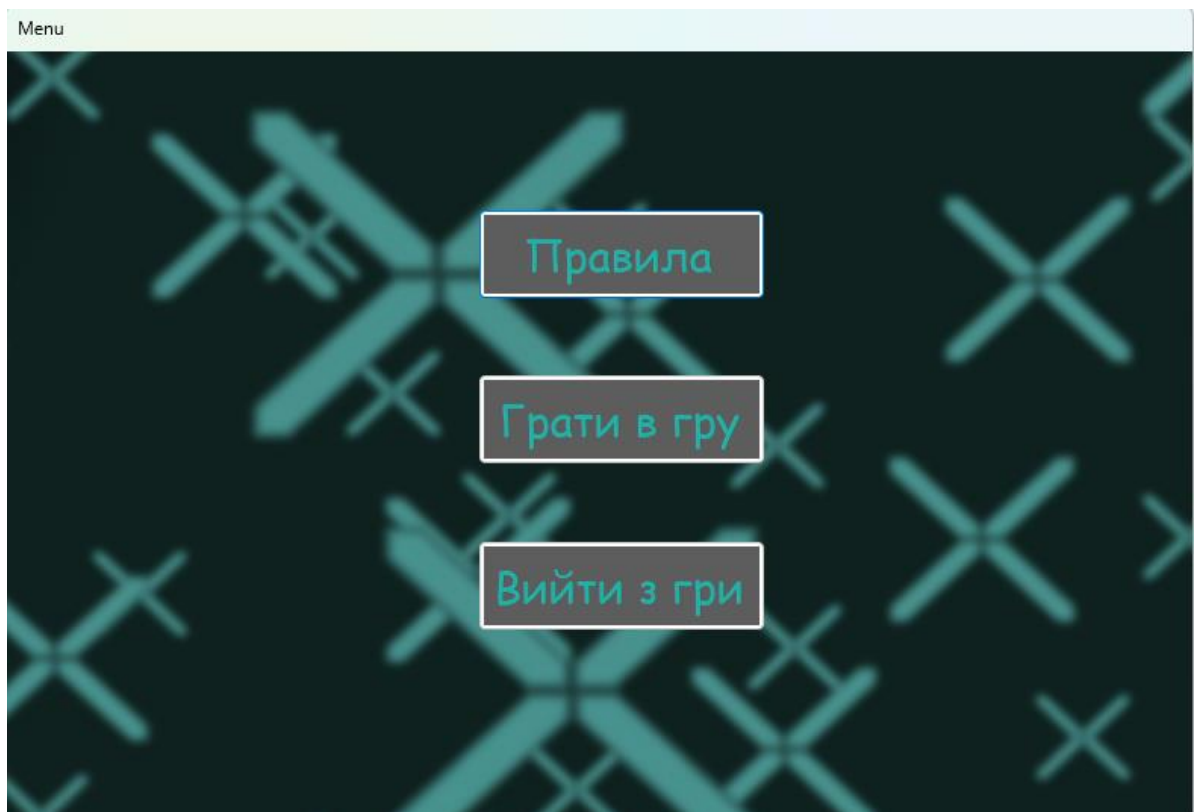
        private void ButGame_Click(object sender, EventArgs e)
        {
            Game GameForm = new Game(); // Створюємо екземпляр форми Game
            GameForm.Show(); // Відображаємо форму Game
            this.Hide(); // Приховуємо поточну форму Rules
        }
    }
}

```

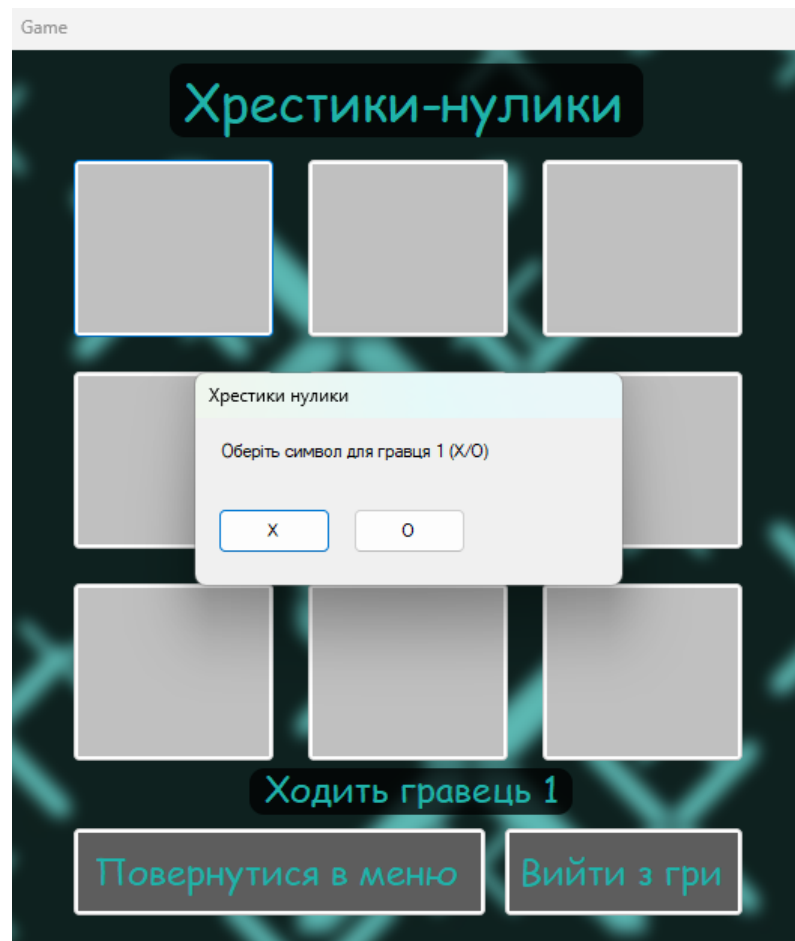
Додаток Б.37 – Ознайомлення з кодом форми правил



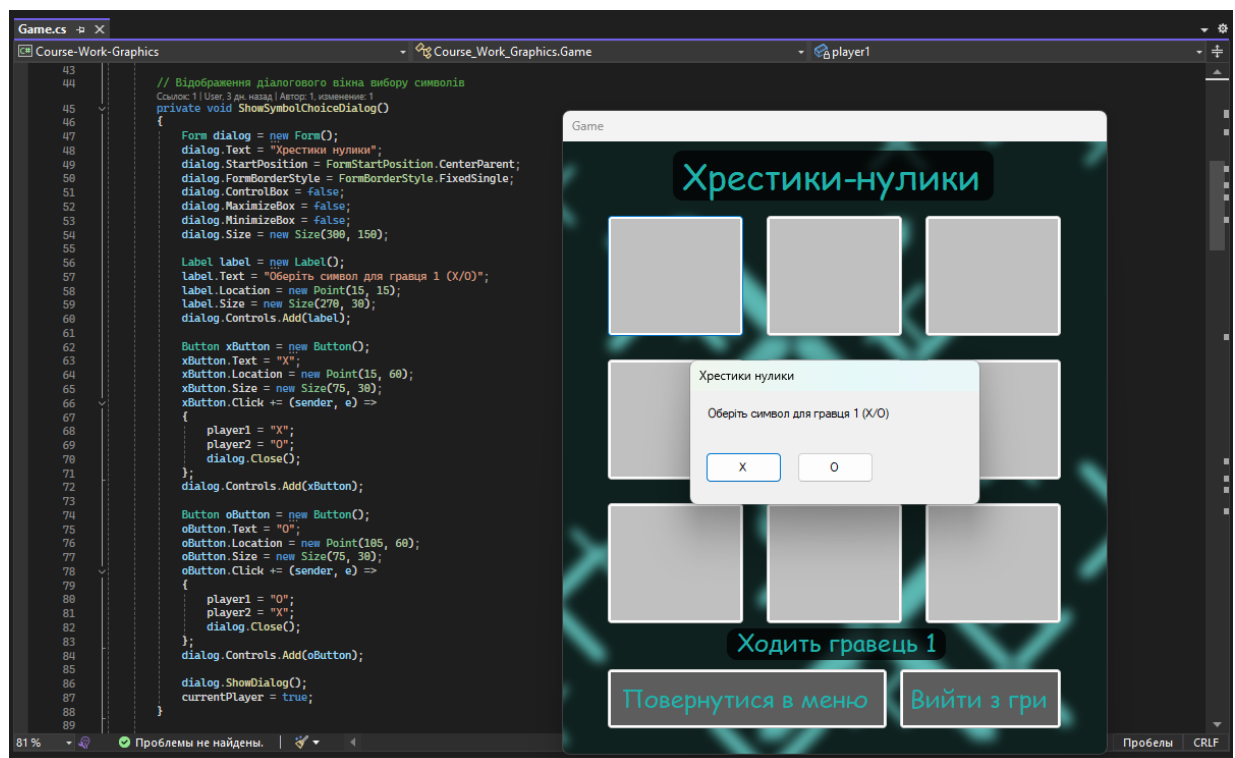
Додаток Б.38 – Запуск форми правил



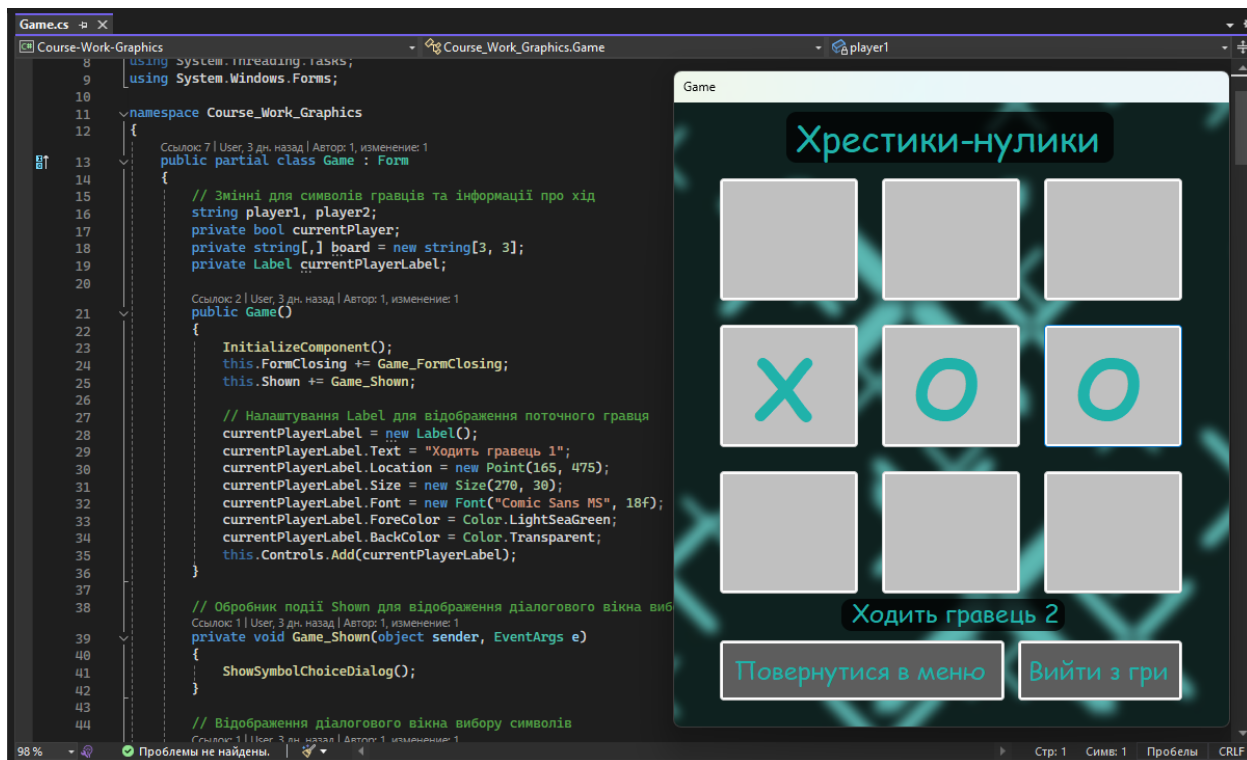
Додаток Б.39 – Перехід з форми правил у форму меню за допомогою кнопки



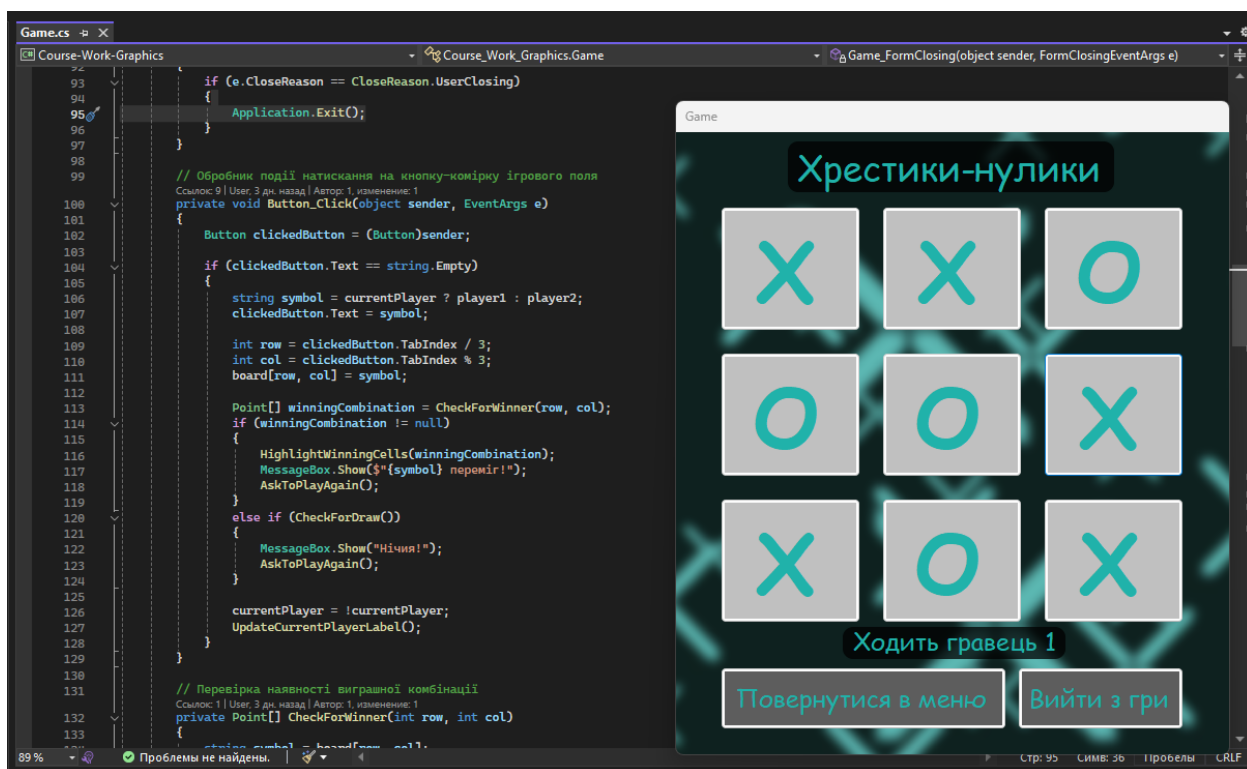
Додаток Б.40 – Перехід з форми правил у форму гри за допомогою кнопки



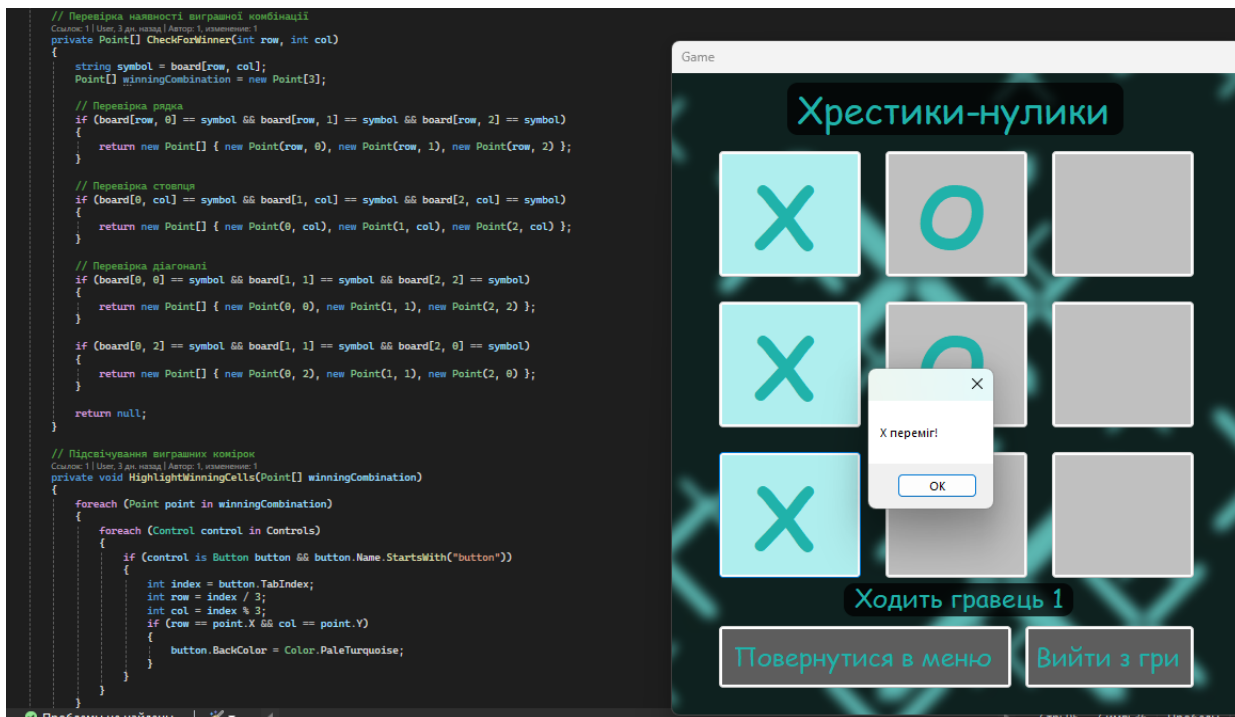
Додаток Б.41 – Перевірка запуску форм на обирання символу для гравця



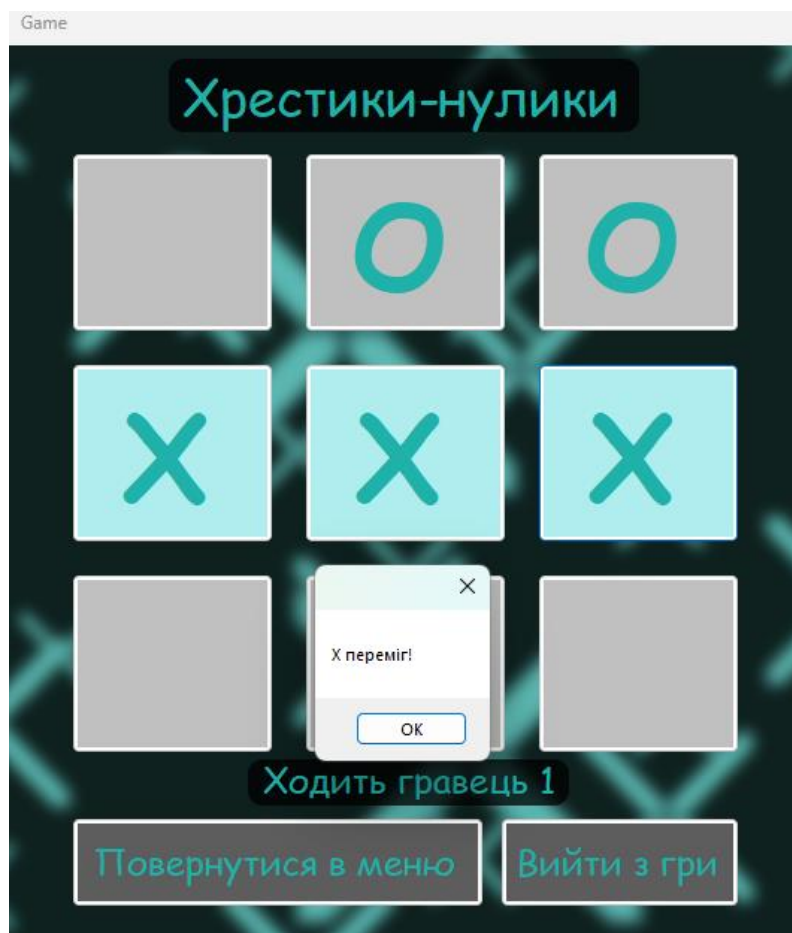
Додаток Б.42 – Перевірка на відображення поточного гравця



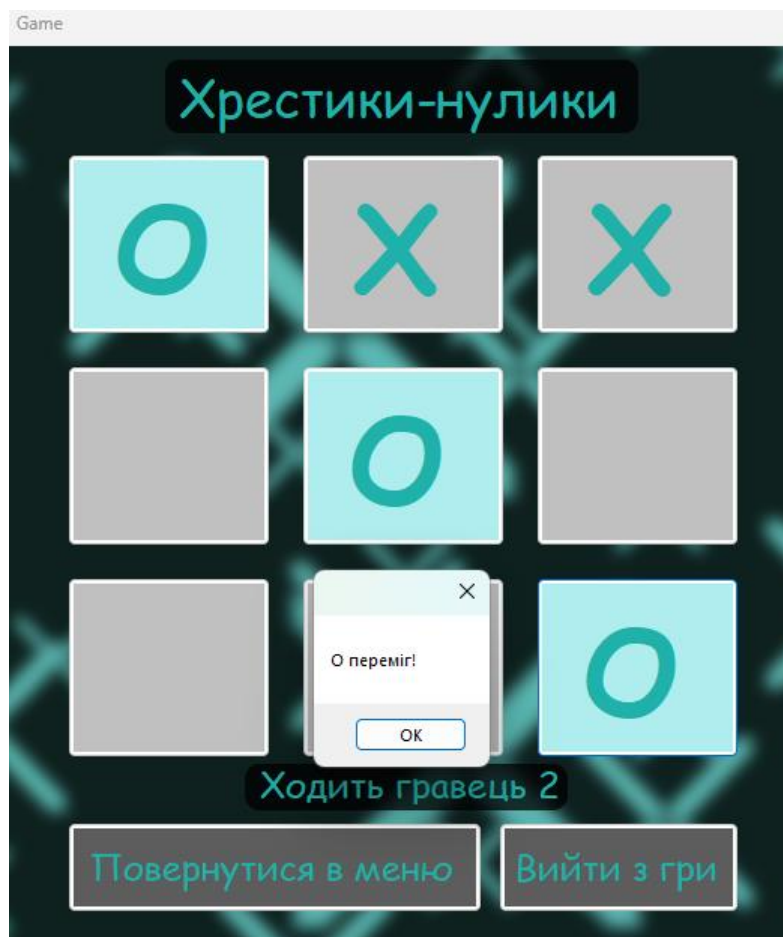
Додаток Б.43 – Перевірка на заповнення всіх комірок



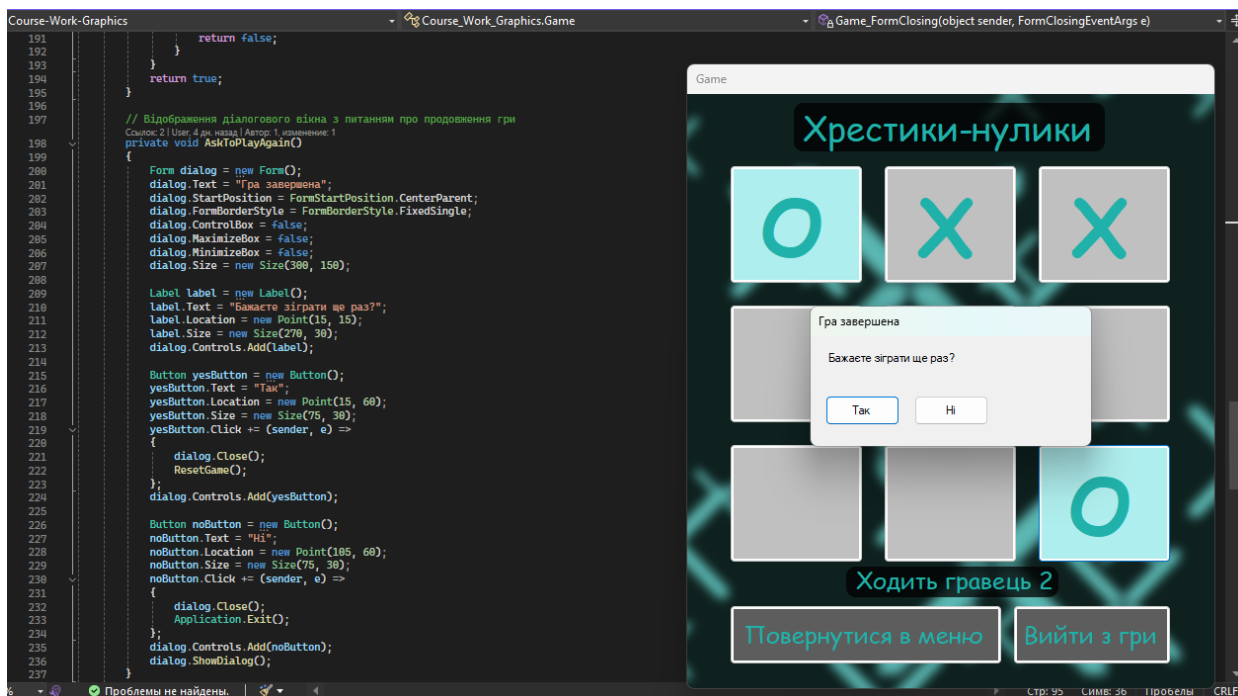
Додаток Б.44 – Перевірка на виграшну комбінації по стовпцям та підсвічування виграшної комбінації



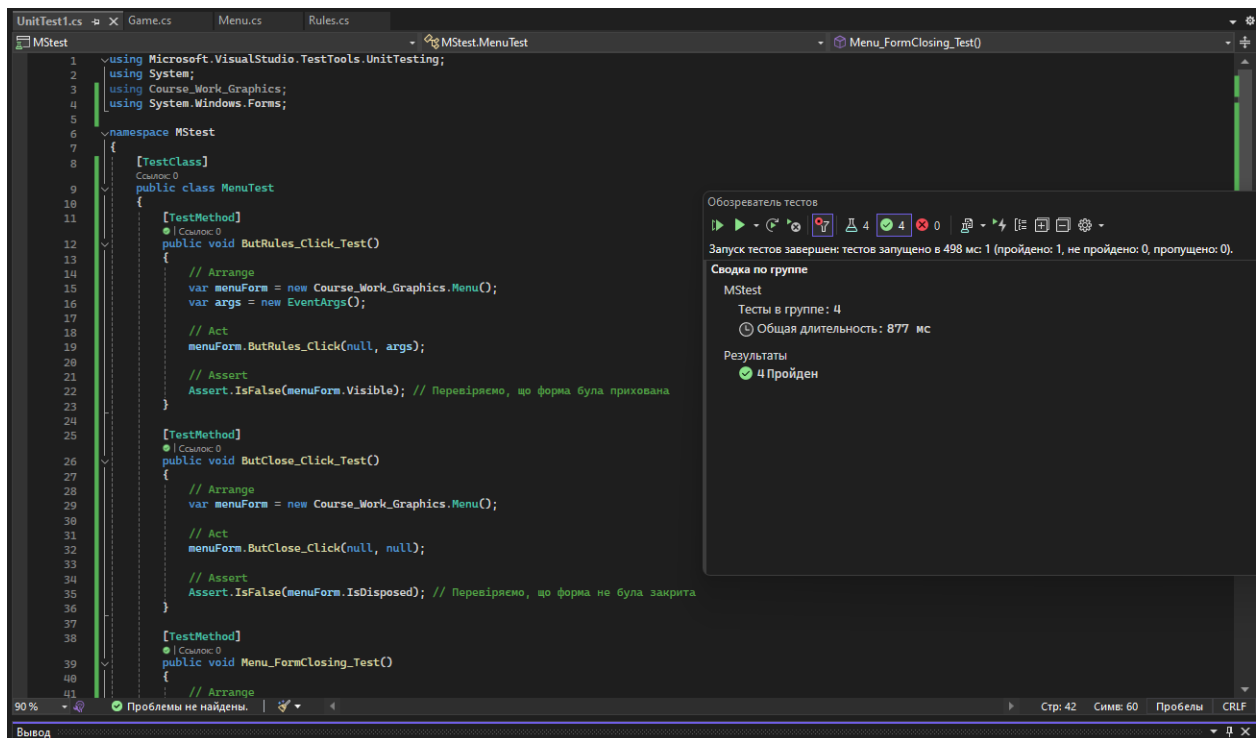
Додаток Б.45 – Перевірка на виграшну комбінацію по рядку



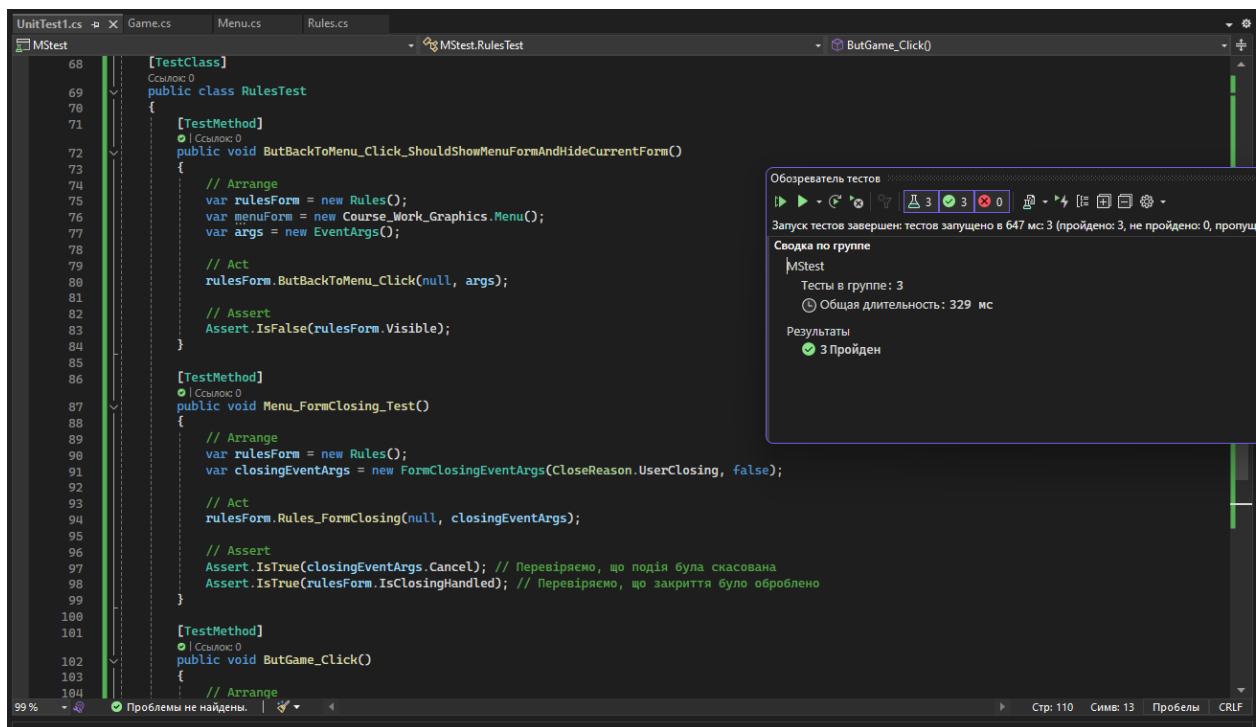
Додаток Б.46 – Перевірка на виграшну комбінацію по діагоналі



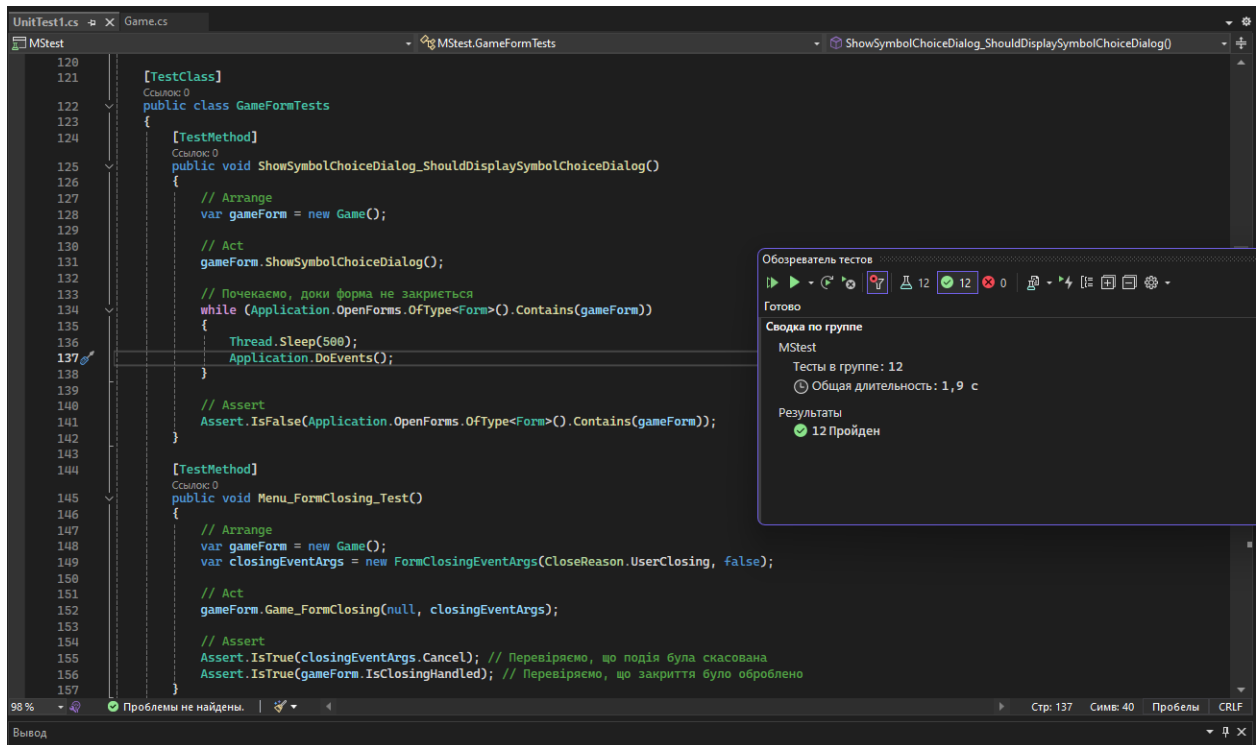
Додаток Б.47 – Перевірка на виведення вікна із запитом продовження гри



Додаток Б.48 – Перевірка на правильність виконання кнопок форми меню

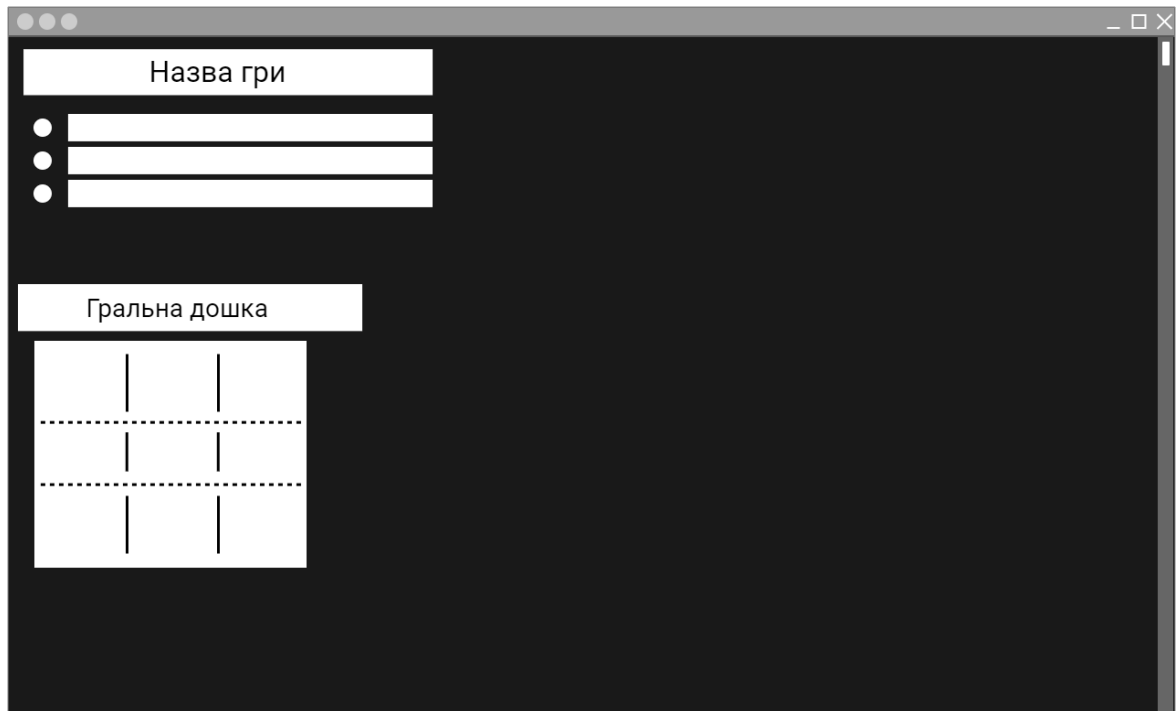


Додаток Б.49 – Перевірка на правильність виконання кнопок форми правил

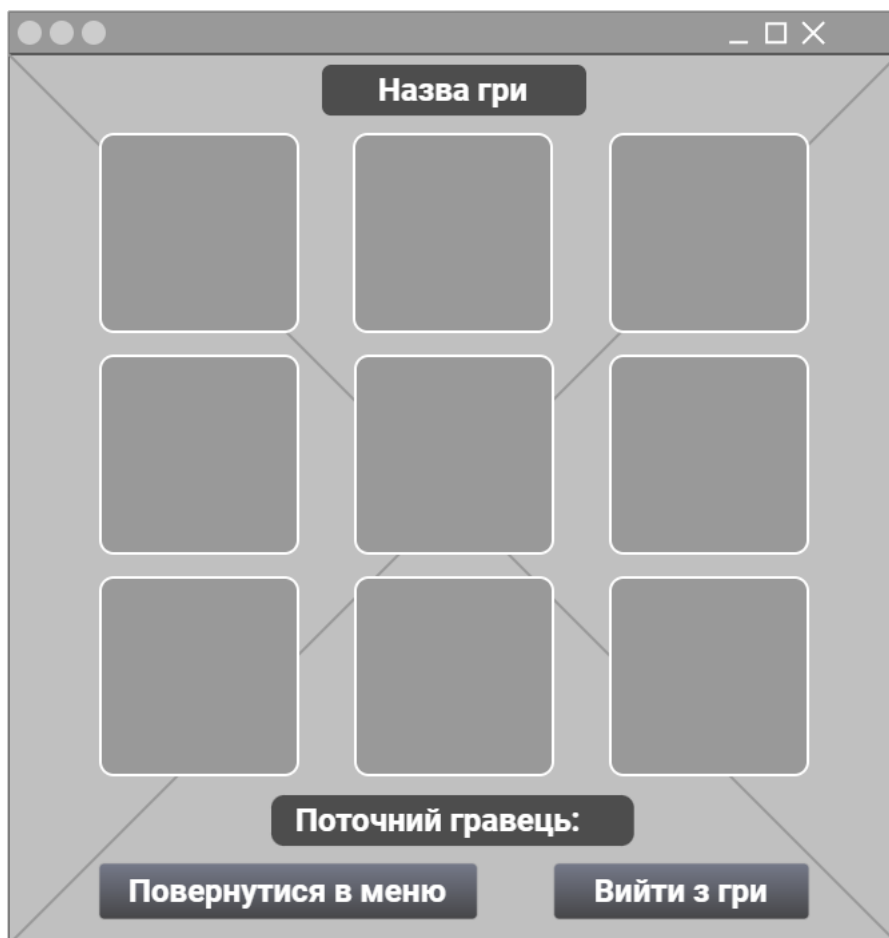


Додаток Б.50 – Перевірка на правильність обробки гри

ДОДАТОК В. ВИМОГИ ДО ІНТЕРФЕЙСУ



Додаток В.1 – Прототип консольного інтерфейсу



Додаток В.2 – Прототип графічного інтерфейсу